TRƯỜNG .....

KHOA .....

NGÀNH .....

BÁO CÁO ĐỒ ÁN

ỨNG DỤNG BÁN QUẦN ÁO

MÔN HỌC: .....

Giảng viên hướng dẫn: .....

Sinh viên thực hiện: .....

MSSV: ..... Lớp: .....

Năm học 2026

MỤC LỤC

1. GIỚI THIỆU 1
1.1. Đặt vấn đề 1
1.2. Mục tiêu và phạm vi 1
1.3. Đối tượng sử dụng 2
1.4. Phương pháp thực hiện 2
1.5. Bố cục báo cáo 3
2. PHÂN TÍCH YÊU CẦU VÀ CHỨC NĂNG 4
2.1. Yêu cầu của đề bài 4
2.2. Ma trận chức năng 4
2.3. Tác nhân và use case 7
2.4. Quy tắc nghiệp vụ 8
2.5. Vòng đời đơn hàng 9
2.6. Phân quyền 10
2.7. Phạm vi loại trừ 11
3. THIẾT KẾ CƠ SỞ DỮ LIỆU 12
3.1. Quy ước dữ liệu 12
3.2. Sơ đồ thực thể liên kết 12
3.3. Từ điển dữ liệu 13
3.4. Sổ cái tồn kho 16
3.5. Các giao dịch then chốt 17
3.6. Giá vốn và lãi gộp 17
3.7. Chỉ mục và truy vấn 18
3.8. Bốn quyết định miền 19
4. THIẾT KẾ GIAO DIỆN 20
4.1. Định hướng thiết kế 20
4.2. Hệ thống thiết kế 20
4.2.1. Màu 20
4.2.2. Chữ 21
4.2.3. Khoảng cách, hình dạng, độ nổi 22
4.2.4. Bộ thành phần dùng chung 22
4.2.5. Trạng thái chung 23
4.3. Điều hướng theo vai trò 23
4.4. Màn hình phía khách 25
4.4.1. Đăng nhập và đăng ký 25
4.4.2. Trang chủ 26
4.4.3. Tìm kiếm và lọc 26

4.4.4. Chi tiết sản phẩm 29
4.4.5. Giỏ hàng 31
4.4.6. Thanh toán và kết quả 33
4.4.7. Đơn hàng 35
4.4.8. Hồ sơ và địa chỉ 36
4.5. Màn hình phía quản trị 38
4.5.1. Tổng quan 38
4.5.2. Sản phẩm 39
4.5.3. Đơn hàng 41
4.5.4. Kho và phiếu nhập 43
4.5.5. Báo cáo 44
4.5.6. Người dùng và danh mục 46
4.6. Khả năng tiếp cận 47
4.7. Đợt đánh giá và những gì đã thay đổi 48
5. CÀI ĐẶT VÀ KIỂM THỬ 49
5.1. Kiến trúc tổng thể 49
5.2. Phía server 50
5.2.1. Công nghệ và phiên bản 50
5.2.2. Luật module 50
5.2.3. Mô hình lỗi 50
5.2.4. Cấu hình và chạy 50
5.3. Hợp đồng API 51
5.3.1. Quy ước 51
5.3.2. Nhóm endpoint 51
5.3.3. Hai cơ chế bảo vệ 52
5.4. Phía Android 52
5.4.1. Kiến trúc và thư viện 52
5.4.2. Tầng mạng và phiên đăng nhập 53
5.4.3. Tầng giao diện 53
5.5. Thanh toán 54
5.5.1. Ranh giới trừu tượng 54
5.5.2. Cổng mock 54
5.5.3. Cổng VNPay 54
5.6. Năm báo cáo 55
5.7. Chiến lược kiểm thử 57
5.7.1. Bốn tầng 57
5.7.2. Cách ly dữ liệu kiểm thử 57
5.7.3. Kiểm thử giao diện 57
5.8. Kết quả lần chạy cuối 58

5.8.1. Dữ liệu seed 58
5.8.2. Kiểm chứng hồ sơ nộp 59
5.9. Hồ sơ nộp 60
6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 61
6.1. Đối chiếu với yêu cầu đề bài 61
6.2. Hạn chế đã biết 62
6.3. Hướng phát triển 63
6.4. Bài học 63
TÀI LIỆU THAM KHẢO 65
Phụ lục A. Danh sách endpoint theo module 66
Phụ lục B. Kịch bản demo và môi trường 70
B.1. Kịch bản demo 70
B.2. Môi trường phát triển 71
B.3. Cách ghi video 71
Phụ lục C. Các bước chạy lại hệ thống 73
C.1. Cơ sở dữ liệu 73
C.2. API 73
C.3. Ứng dụng Android 73
C.4. Tài khoản demo 74
C.5. Chạy kiểm thử 74
C.6. Dựng lại hồ sơ 74

1. GIỚI THIỆU

1.1. Đặt vấn đề

Bán lẻ quần áo tại Việt Nam đã chuyển phần lớn khâu chọn hàng sang điện thoại: khách xem ảnh, so kích cỡ, hỏi giá và đặt hàng trên thiết bị di động, trong khi cửa hàng vẫn phải quản lý tồn kho theo từng kích cỡ và màu, theo dõi đơn giao và biết chính xác lãi gộp của từng đơn. Hai phía này thường được giải quyết bằng hai hệ thống rời nhau: một website bán hàng cho khách và một file bảng tính cho chủ cửa hàng. Cách làm đó dẫn tới ba hệ quả quen thuộc: số lượng tồn trên màn hình khách không khớp với số lượng thực tế trong kho, đơn hàng bị xác nhận hai lần cho cùng một lượng hàng, và báo cáo lãi gộp chỉ được dựng lại bằng tay vào cuối tháng.

Đồ án này xây dựng **Stylo**, một ứng dụng bán quần áo gồm ứng dụng Android cho khách mua hàng và phần quản trị nằm trong chính ứng dụng đó, dùng chung một API và một cơ sở dữ liệu. Điểm khác biệt của hướng tiếp cận không nằm ở giao diện mà ở chỗ mọi con số bán hàng đều neo vào hai thứ được lưu vết: tồn kho là sổ cái chỉ ghi thêm (append-only ledger), và doanh thu chỉ được ghi nhận khi đơn đã thanh toán. Nhờ vậy khách thấy đúng số lượng còn bán, còn chủ cửa hàng đọc được cùng một con số ở màn hình báo cáo và ở cơ sở dữ liệu.

1.2. Mục tiêu và phạm vi

Mục tiêu của đồ án là một hệ thống chạy được đầu-cuối trên máy phát triển, phủ đủ các chức năng mà đề bài yêu cầu, với hai vai trò người dùng và thanh toán trực tuyến ở mức sandbox:

- **Khách (customer)**: tìm kiếm và lọc sản phẩm, xem chi tiết theo biến thể, quản lý giỏ hàng, đặt

hàng, thanh toán khi nhận hàng (COD) hoặc thanh toán online qua cổng VNPay ở mức sandbox, theo dõi và huỷ đơn trong giới hạn trạng thái cho phép.

- **Quản trị (admin)**: quản lý sản phẩm, biến thể và ảnh; quản lý danh mục và thương hiệu; xử lý đơn

theo đúng vòng đời; tạo phiếu nhập kho nhiều dòng và lưu lịch sử giá nhập; quản lý người dùng; xem năm báo cáo doanh thu, sản phẩm bán chạy, đơn theo trạng thái, tồn thấp và chi phí–lãi gộp.

- **Khách vãng lai (guest)**: xem và tìm sản phẩm mà không cần tài khoản; giỏ hàng và đơn hàng yêu cầu

đăng nhập.

Phạm vi được chốt hẹp có chủ đích để phần đã làm là phần chạy được: một ứng dụng Android duy nhất cho cả hai vai trò (không có web admin riêng), một cơ sở dữ liệu MySQL, backend chạy nội bộ trong mạng LAN của máy phát triển, không triển khai lên máy chủ công cộng. Những phần không thuộc phạm vi — khuyến mãi, voucher, đánh giá, danh sách yêu thích, đổi trả và hoàn tiền, đa tiền tệ — được liệt kê ở mục 2.7 để người đọc không kỳ vọng nhầm.

1.3. Đối tượng sử dụng

Vai trò	Mô tả	Việc chính trong hệ thống
Khách vãng lai	Người chưa đăng nhập, không phải một bản ghi trong cơ sở dữ liệu	Xem trang chủ, tìm kiếm, lọc, xem chi tiết sản phẩm
Khách hàng	Tài khoản `CUSTOMER` đang hoạt động	Giỏ hàng, đặt hàng, thanh toán, địa chỉ, hồ sơ, đơn của mình
Quản trị	Tài khoản `ADMIN` do seed tạo, không tự đăng ký	Sản phẩm, biến thể, ảnh, danh mục, thương hiệu, đơn, kho, người dùng, báo cáo

1.4. Phương pháp thực hiện

Đồ án được làm theo hướng đặc tả trước, mã sau. Trước khi viết dòng mã đầu tiên, các quyết định lớn được ghi thành văn bản: phạm vi chức năng, mô hình dữ liệu, hợp đồng API, thiết kế giao diện, chiến lược kiểm thử và công thức giá vốn. Mười bảy quyết định kiến trúc (ADR) lưu trong `docs/adr/`, mỗi quyết định nêu bối cảnh, lựa chọn và hệ quả; nhờ vậy khi một lựa chọn bị thay đổi, tài liệu chỉ rõ chỗ nào phải sửa theo.

Cách làm này có giá: văn bản phải cập nhật cùng mã, nếu không nó nhanh chóng trở thành tài liệu sai. Đổi lại, ba thứ đắt nhất trong một dự án nhỏ đều có chỗ dựa: API

không tự mọc thêm trường tuỳ hứng, giao diện không tự đặt màu ngoài bảng token, và mỗi chức năng có đường kiểm chứng rõ ràng — hoặc một kiểm thử tích hợp ở backend, hoặc một hành trình Kaspresso trên máy ảo, hoặc cả hai.

Ba tầng kiểm chứng được dùng xuyên suốt: kiểm thử đơn vị cho phần logic thuần, kiểm thử tích hợp cho đường HTTP và giao dịch cơ sở dữ liệu, và kiểm thử giao diện tự động cho các luồng đã hoàn thiện. Số liệu của lần chạy cuối cùng được ghi ở chương 5, kèm môi trường chạy để người đọc đối chiếu.

1.5. Bố cục báo cáo

Báo cáo gồm sáu chương và ba phụ lục:

- **Chương 1** nêu bối cảnh, mục tiêu, phạm vi và phương pháp.
- **Chương 2** phân tích yêu cầu: mười hai nhóm chức năng, tác nhân, use case, quy tắc nghiệp vụ và
vòng đời đơn hàng.
- **Chương 3** trình bày thiết kế cơ sở dữ liệu: mười sáu bảng, quy ước dữ liệu, sổ cái tồn kho và
bốn quyết định miền có ADR riêng.
- **Chương 4** trình bày thiết kế giao diện: hệ thống thiết kế, điều hướng theo vai trò, các màn hình
của khách và của quản trị, cùng ảnh chụp thật từ máy ảo.
- **Chương 5** trình bày cài đặt và kiểm thử: kiến trúc hai phía, hợp đồng API, luồng thanh toán,
chiến lược kiểm thử và số liệu của lần chạy cuối.
- **Chương 6** kết luận, nêu giới hạn đã biết và hướng phát triển.
- **Phụ lục A** liệt kê toàn bộ endpoint theo module; **Phụ lục B** là kịch bản demo và môi trường;
Phụ lục C là các bước chạy lại hệ thống từ đầu.

2. PHÂN TÍCH YÊU CẦU VÀ CHỨC NĂNG

2.1. Yêu cầu của đề bài

Đề bài yêu cầu phân tích chức năng, thiết kế cơ sở dữ liệu, thiết kế giao diện và xây dựng một ứng dụng bán quần áo có các chức năng: đăng ký/đăng nhập; quản lý thông tin cá nhân; quản lý giỏ hàng (thêm, sửa, xoá); quản lý sản phẩm (thêm, sửa, xoá); quản lý đơn hàng; quản lý người dùng; tìm kiếm sản phẩm theo danh mục, tên, màu sắc, giá và thương hiệu; chi tiết sản phẩm; thanh toán; báo cáo và thống kê; phiếu giá nhập kho sản phẩm. Môi trường được chỉ định: Kotlin cho Android, Node.js cho backend, MySQL cho cơ sở dữ liệu.

Mười một mục trên là yêu cầu tối thiểu. Khi phân tích, hai điểm cần làm rõ trước khi viết mã:

- **Ranh giới giữa khách vãng lai và khách hàng.** Đề bài không nói ai được xem sản phẩm. Nếu bắt đăng

nhập mới xem được hàng thì ứng dụng bán lẻ mất đi lợi thế lớn nhất của nó; nếu cho thêm giỏ hàng khi chưa đăng nhập thì phải giải quyết bài toán gộp giỏ giữa các thiết bị — một việc không thuộc phạm vi đồ án. Ranh giới được chốt: xem và tìm kiếm không cần tài khoản, giỏ hàng và đơn hàng thì cần.

- **Quản trị đặt ở đâu.** Đề bài yêu cầu năm chức năng quản trị. Một website quản trị riêng sẽ nhân đôi

khối lượng giao diện và không có thời gian làm đúng. Quản trị được đặt trong cùng ứng dụng Android, chỉ mở khi tài khoản có vai trò `ADMIN`, dùng chung tầng mạng và tầng dữ liệu với phía khách.

2.2. Ma trận chức năng

Mười hai nhóm chức năng dưới đây là bản chốt phạm vi của đồ án; mỗi dòng nêu tác nhân, mức hoàn thiện và tiêu chí nghiệm thu. Cột "Màn hình" chỉ nơi chức năng xuất hiện trong ứng dụng.

Bảng 2.1. Ma trận mười hai nhóm chức năng và tiêu chí nghiệm thu

#	**Chức năng**	**Tác nhân**	**Tiêu chí nghiệm thu**	**Màn hình**
1	Đăng ký, đăng nhập	Khách hàng	Email + mật khẩu; access token 15 phút, refresh 30 ngày; sai mật khẩu trả một lỗi chung; không xác thực email, không quên mật khẩu	Đăng nhập / Đăng ký
2	Hồ sơ và địa chỉ	Khách hàng	Sửa họ tên, số điện thoại; thêm, sửa, xoá địa chỉ; đúng một địa chỉ mặc định; không có ảnh đại diện	Tôi, Địa chỉ giao hàng
3	Giỏ hàng	Khách hàng	Chỉ khi đã đăng nhập; thêm, sửa số lượng, xoá dòng, xoá tất cả; số lượng không vượt tồn; tổng tiền và phí vận chuyển tính ở server	Giỏ hàng
4	Quản lý sản phẩm	Quản trị	Tạo, sửa, ẩn sản phẩm; biến thể size × màu có SKU riêng; tối đa 5 ảnh upload; ẩn thay vì xoá khi đã phát sinh đơn	Sản phẩm (quản trị)
5	Quản lý đơn hàng	Khách hàng, Quản trị	Khách xem danh sách, chi tiết, huỷ khi đơn còn `PENDING` hoặc `CONFIRMED`; quản trị chỉ chuyển theo bảng chuyển trạng thái hợp lệ; huỷ đơn hoàn tồn	Đơn hàng, Chi tiết đơn

#	Chức năng	Tác nhân	Tiêu chí nghiệm thu	Màn hình
6	Quản lý người dùng	Quản trị	Danh sách, tìm theo tên hoặc email, khoá và mở khoá; không đổi vai trò trong ứng dụng; tài khoản quản trị tạo bằng seed	Người dùng
7	Tìm kiếm sản phẩm	Khách vãng lai, Khách hàng	Từ khoá không dấu vẫn khớp; lọc kết hợp danh mục, thương hiệu, màu, kích cỡ, khoảng giá; sắp xếp theo giá hoặc mới nhất; phân trang	Tìm kiếm
8	Chi tiết sản phẩm	Khách vãng lai, Khách hàng	Ảnh, mô tả, chọn biến thể kèm tồn và giá theo biến thể; chọn số lượng trong giới hạn tồn; thêm vào giỏ đúng biến thể đã chọn	Chi tiết sản phẩm
9	Thanh toán	Khách hàng	COD hoặc thanh toán online qua VNPay sandbox, có cổng mock cho môi trường phát triển; giao dịch có trạng thái `PENDING`, `SUCCEEDED`, `FAILED`; `paid_at` là nguồn sự thật duy nhất	Thanh toán, Kết quả thanh toán
10	Báo cáo và thống kê	Quản trị	Năm báo cáo: doanh thu, sản phẩm bán chạy, đơn theo trạng thái, tồn thấp, chi phí và lãi gộp; lọc theo khoảng ngày; không xuất tệp	Báo cáo

#	Chức năng	Tác nhân	Tiêu chí nghiệm thu	Màn hình
11	Phiếu nhập kho	Quản trị	Tạo phiếu nhiều dòng (biến thể, số lượng, đơn giá nhập); cộng tồn trong cùng giao dịch; phiếu bất biến sau khi lưu	Kho
12	Danh mục và thương hiệu	Quản trị	Tạo, sửa, ẩn danh mục và thương hiệu; không xoá danh mục còn sản phẩm	Danh mục & thương hiệu

Ba ranh giới nghiệp vụ trong bảng trên đáng được nêu riêng vì chúng quyết định phần lớn thiết kế:

Thứ nhất, **tồn kho gắn với biến thể, không gắn với sản phẩm**. Một chiếc áo có ba kích cỡ và hai màu là sáu đơn vị bán độc lập, mỗi đơn vị một SKU, một giá và một số lượng tồn. Nếu tồn kho đặt ở mức sản phẩm thì không thể trả lời câu hỏi thực tế nhất của cửa hàng: còn chiếc size M màu trắng hay không.

Thứ hai, **sản phẩm đã phát sinh đơn không bị xoá**. Xoá cứng sẽ làm đơn cũ mất tham chiếu tới mặt hàng; vì vậy trạng thái hiển thị là một cột (`ACTIVE`/`INACTIVE`), và xoá cứng chỉ xảy ra khi chưa có dòng nào tham chiếu tới sản phẩm.

Thứ ba, **huỷ đơn phải hoàn tồn trong cùng giao dịch**. Đây là chỗ dễ sinh sai lệch nhất giữa số liệu bán hàng và số liệu kho, nên nó được thiết kế ở tầng cơ sở dữ liệu (chương 3) và kiểm thử ở tầng tích hợp (chương 5).

2.3. Tác nhân và use case

Ba tác nhân và tập use case chính được liệt kê ở bảng dưới. Các use case quản trị đều yêu cầu vai trò `ADMIN`; các use case mua hàng yêu cầu vai trò `CUSTOMER`; hai use case xem và tìm kiếm không yêu cầu đăng nhập.

Bảng 2.2. Danh sách use case theo tác nhân

Tác nhân	Use case
Khách vãng lai	Xem trang chủ; tìm kiếm theo từ khoá; lọc và sắp xếp; xem chi tiết sản phẩm; mở màn hình đăng nhập khi cần

Tác nhân	Use case
Khách hàng	Đăng ký; đăng nhập; đăng xuất; sửa hồ sơ; quản lý địa chỉ; thêm vào giỏ; sửa số lượng; xoá dòng; đặt hàng COD; đặt hàng và thanh toán online; xem danh sách đơn; xem chi tiết đơn; huỷ đơn
Quản trị	Xem tổng quan; tạo và sửa sản phẩm; quản lý biến thể; tải ảnh sản phẩm; ẩn/hiện sản phẩm; quản lý danh mục và thương hiệu; xử lý đơn theo vòng đời; tạo phiếu nhập kho; quản lý người dùng; xem năm báo cáo

Luồng chuẩn của khách hàng, dùng làm kịch bản demo ở phụ lục B, gồm: tìm sản phẩm bằng từ khoá không dấu, lọc theo thương hiệu, mở chi tiết, chọn màu và kích cỡ, thêm vào giỏ, đặt hàng COD với địa chỉ mặc định, rồi theo dõi đơn ở màn hình Đơn hàng. Luồng quản trị gồm: xem tổng quan, xác nhận một đơn đang chờ, tạo sản phẩm mới, tạo phiếu nhập kho và đọc báo cáo tháng.

2.4. Quy tắc nghiệp vụ

Các quy tắc dưới đây được kiểm thử trực tiếp ở tầng service và tầng cơ sở dữ liệu, không chỉ được mô tả trong tài liệu.

Bảng 2.3. Quy tắc nghiệp vụ bắt buộc

Nhóm	Quy tắc	Hệ quả khi vi phạm
Tài khoản	Một tài khoản có đúng một vai trò; tài khoản quản trị chỉ sinh ra từ seed	Đăng nhập bị từ chối, không có đường tự nâng quyền
Tài khoản	Tài khoản bị khoá không đăng nhập được, kể cả khi token cũ còn hạn	Request trả `ACCOUNT_DISABLED`
Địa chỉ	Mỗi khách có đúng một địa chỉ mặc định; địa chỉ đầu tiên tự thành mặc định	Service đặt lại cờ trong cùng giao dịch
Giỏ hàng	Số lượng một dòng không vượt tồn hiện tại của biến thể	Thêm hoặc sửa vượt tồn trả `OUT_OF_STOCK`
Giỏ hàng	Tổng tiền luôn do server tính từ giá hiện hành của biến thể	Client không gửi giá lên khi đặt hàng
Đặt hàng	Phí vận chuyển một mức phẳng, miễn khi tạm tính đạt ngưỡng 500.000 ₫; server tính lại lúc đặt và lưu vào đơn	Đổi biến môi trường không làm đổi đơn đã đặt
Đặt hàng	Một biến thể chỉ được bán khi sản phẩm đang `ACTIVE` và biến thể còn hàng	`ITEM_UNAVAILABLE` hoặc `OUT_OF_STOCK`

Nhóm	Quy tắc	Hệ quả khi vi phạm
Đặt hàng	Mỗi lần bấm đặt hàng mang một `clientOrderId`; gửi lại cùng mã trả về đơn đã tạo	Không tạo đơn trùng khi mạng chập chờn
Tồn kho	Mọi thay đổi tồn đi kèm một dòng sổ cái `StockMovement` trong cùng giao dịch	Tồn và sổ cái luôn khớp
Đơn hàng	Chỉ chuyển trạng thái theo bảng ở mục 2.5; trạng thái cuối là `DELIVERED` và `CANCELLED`	`INVALID_ORDER_STATE`
Đơn hàng	Huỷ đơn chỉ khi đơn chưa thanh toán (`paid_at` rỗng)	Không có luồng hoàn tiền
Thanh toán	Đơn COD được đánh dấu đã thanh toán khi chuyển sang `DELIVERED`; đơn online khi cổng xác nhận	Doanh thu chỉ ghi nhận trên đơn đã thanh toán
Thanh toán	Một đơn có nhiều lần thử thanh toán, mỗi lần một bản ghi `Payment` riêng	Lần thử hỏng không xoá lần thử trước
Sản phẩm	Ảnh sản phẩm tải lên kiểm tra magic byte, tối đa 5 ảnh mỗi sản phẩm	Tệp sai định dạng bị từ chối
Danh mục	Không xoá danh mục hoặc thương hiệu còn sản phẩm tham chiếu	Khoá ngoại `RESTRICT` chặn ở tầng dữ liệu
Phiếu nhập	Phiếu đã lưu là bất biến; sai sót xử lý bằng phiếu mới	Không có endpoint sửa hoặc xoá phiếu

2.5. Vòng đời đơn hàng

Đơn hàng có năm trạng thái. `PENDING` là trạng thái sau khi đặt; `CONFIRMED` là đã được cửa hàng xác nhận; `SHIPPING` là đang giao; `DELIVERED` và `CANCELLED` là hai trạng thái kết thúc. Mỗi chuyển trạng thái đi qua một hàm service duy nhất, hàm này kiểm tra cặp trạng thái, người thực hiện và điều kiện thanh toán trước khi ghi.

Bảng 2.4. Các chuyển trạng thái hợp lệ của đơn hàng

Từ	Sang	Ai được làm	Tác dụng phụ
`PENDING`	`CONFIRMED`	Quản trị, hoặc cổng thanh toán khi thanh toán thành công	Đơn online: ghi `paid_at`
`PENDING`	`CANCELLED`	Khách sở hữu đơn, quản trị	Hoàn tồn, ghi sổ cái
`CONFIRMED`	`SHIPPING`	Quản trị	Không

Từ	Sang	Ai được làm	Tác dụng phụ
CONFIRMED	CANCELLED	Khách sở hữu đơn, quản trị	Hoàn tồn, ghi sổ cái
SHIPPING	DELIVERED	Quản trị	Đơn COD: ghi paid_at

Hai trạng thái kết thúc không có đường ra. Quy tắc chặn thứ hai là điều kiện thanh toán: đơn đã có paid_at không thể huỷ, vì hệ thống không có luồng hoàn tiền. Với đơn COD, điều này không cản trở gì trong thực tế: khách chưa trả tiền cho tới khi nhận hàng, nên việc huỷ trước khi giao vẫn hợp lệ; sau khi giao thì đơn đã thanh toán và không huỷ được nữa.

Một chi tiết dễ bỏ sót: **ngày ghi nhận doanh thu của đơn COD là ngày giao hàng, không phải ngày đặt**. Báo cáo doanh thu vì vậy đọc theo paid_at, và một đơn đặt ngày 5 nhưng giao ngày 8 sẽ xuất hiện trong báo cáo của ngày 8. Đây là lựa chọn có chủ đích, được ghi thành ADR-0007 và được kiểm thử ở nhóm báo cáo.

2.6. Phân quyền

Ứng dụng dùng một bảng người dùng và một cột vai trò; không có bảng quyền riêng. Mọi endpoint được chia thành bốn mức: công khai, yêu cầu đăng nhập, yêu cầu vai trò quản trị, và công khai có giới hạn (chỉ đọc dữ liệu đang bán). Bảng dưới là ánh xạ mức quyền cho từng nhóm endpoint.

Bảng 2.5. Ánh xạ quyền theo nhóm endpoint

Nhóm	Mức truy cập	Ghi chú
Xác thực, hồ sơ, địa chỉ	Công khai với đăng ký và đăng nhập; còn lại yêu cầu token	Token hết hạn được làm mới bằng refresh token
Danh mục công khai, sản phẩm, bộ lọc	Công khai	Chỉ trả sản phẩm ACTIVE
Giỏ hàng	Yêu cầu đăng nhập	Giỏ thuộc về chính người gọi
Đơn hàng của tôi, thanh toán	Yêu cầu đăng nhập	Truy vấn luôn kèm điều kiện chủ sở hữu
Quản trị sản phẩm, danh mục, thương hiệu, ảnh	Yêu cầu vai trò ADMIN	
Quản trị đơn, kho, người dùng, báo cáo	Yêu cầu vai trò ADMIN	

Kiểm tra quyền nằm ở middleware, không nằm trong từng handler: một middleware xác thực token và nạp người dùng (từ chối tài khoản bị khoá), một middleware kiểm tra vai trò. Cách này khiến việc thêm endpoint mới khó quên kiểm tra quyền hơn so với kiểm tra rải rác.

2.7. Phạm vi loại trừ

Những mục sau được chủ động loại khỏi phạm vi, và được nêu ở đây để người đọc không tìm kiếm chúng trong chương trình:

- Giỏ hàng cho khách vãng lai và gộp giỏ giữa các thiết bị.
- Quên mật khẩu, đổi mật khẩu qua email, xác thực email.
- Đổi trả hàng, hoàn tiền, trạng thái trả hàng; đơn đã thanh toán không huỷ được.
- Đổi vai trò người dùng trong ứng dụng; tài khoản quản trị chỉ có từ seed.
- Sửa hoặc xoá phiếu nhập kho sau khi lưu.
- Xuất báo cáo ra CSV hoặc Excel; ảnh đại diện người dùng.
- Khuyến mãi, voucher, đánh giá sản phẩm, danh sách yêu thích, đa tiền tệ.

Việc loại trừ không làm giảm tính đầy đủ của đồ án: mười một chức năng đề bài yêu cầu đều nằm trong phạm vi và đều có đường kiểm chứng ở chương 5.

3. THIẾT KẾ CƠ SỞ DỮ LIỆU

3.1. Quy ước dữ liệu

Cơ sở dữ liệu gồm mười sáu bảng, đủ cho mười hai nhóm chức năng ở chương 2. Trước khi liệt kê từng bảng, các quy ước dưới đây áp dụng cho toàn bộ lược đồ; chúng được chọn một lần để mọi bảng trông giống nhau, thay vì mỗi bảng theo một kiểu.

Bảng 3.1. Quy ước chung của lược đồ

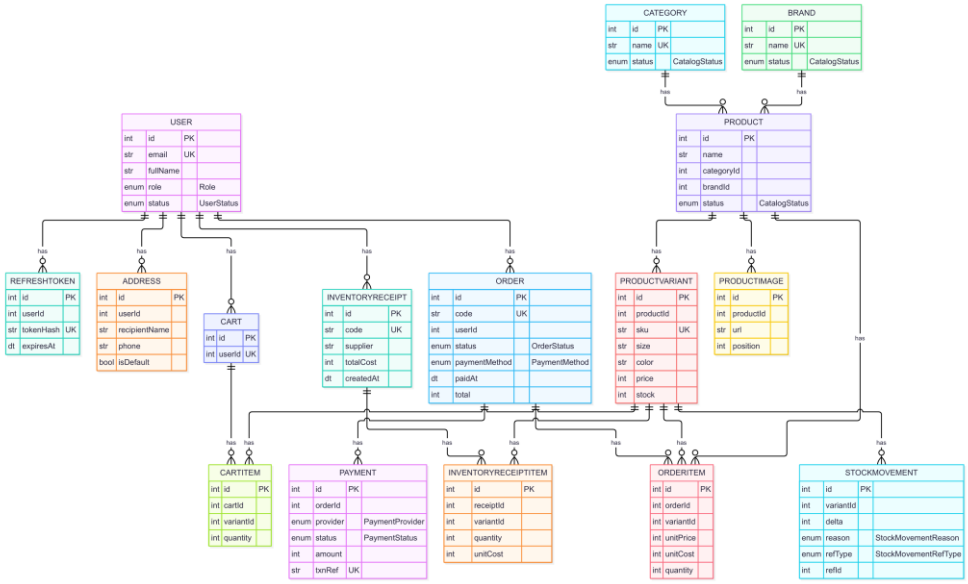
Quy tắc	Giá trị
Engine và bảng mã	InnoDB, `utf8mb4`, collation `utf8mb4_0900_ai_ci` (không phân biệt hoa thường và dấu)
Tên bảng	`snake_case` số ít, ánh xạ qua `@@map`; tên model `PascalCase`
Tên cột	`snake_case` qua `@map`; trường Prisma `camelCase`
Khoá chính	`id INT AUTO_INCREMENT`, không dùng UUID
Thời gian	`created_at`/`updated_at` kiểu `DATETIME(3)`, lưu UTC; ứng dụng hiển thị theo `Asia/Ho_Chi_Minh`
Tiền	`INT` đơn vị VND, không phần thập phân
Enum	`ENUM` gốc của MySQL, giá trị `SCREAMING_SNAKE_CASE`
Ẩn hay xoá	Hiển thị bằng cột `status`; xoá cứng chỉ khi không còn dòng nào tham chiếu (`RESTRICT`)
Từ khoá trùng	`order` là từ khoá của MySQL, nên mọi câu SQL thô phải viết `` `order` ``

Bốn quy ước quan trọng nhất có ADR riêng vì chúng ảnh hưởng tới nhiều tầng: thanh toán theo từng lần thử (ADR-0001), sổ cái tồn kho và việc trừ tồn lúc đặt hàng (ADR-0002), tiền là số nguyên VND (ADR-0003), và đơn hàng lưu ảnh chụp dữ liệu tại thời điểm đặt (ADR-0004). Bốn quyết định này được tóm tắt ở mục 3.8.

3.2. Sơ đồ thực thể liên kết

Sơ đồ dưới đây được sinh trực tiếp từ `server/prisma/schema.prisma` bằng một script đọc lược đồ, nên nó không thể lệch khỏi cơ sở dữ liệu đang chạy: đổi tên

một cột trong lược đồ là bước sinh lại sơ đồ báo lỗi ngay. Bản in A3 nộp kèm là cùng một sơ đồ, phóng to.



Hình 3.1. Sơ đồ thực thể liên kết của mười sáu bảng, sinh từ lược đồ Prisma

Sơ đồ chia thành bốn cụm theo tự nhiên. Cụm tài khoản gồm `user`, `refresh_token`, `address` và `cart`. Cụm danh mục gồm `category`, `brand`, `product`, `product_variant` và `product_image`. Cụm bán hàng gồm `order`, `order_item` và `payment`. Cụm kho gồm `inventory_receipt`, `inventory_receipt_item` và `stock_movement`. Hai đường nối đáng chú ý: `order_item` trỏ tới cả `order` và `product_variant`, và `stock_movement` trỏ tới `product_variant` cùng một cặp (`ref_type`, `ref_id`) trỏ về chứng từ sinh ra nó. Cặp này là một tham chiếu mềm có chủ đích, vì một dòng sổ cái có thể đến từ đơn hàng hoặc từ phiếu nhập kho, và khoá ngoại cứng sẽ buộc phải tách bảng.

3.3. Từ điển dữ liệu

Mười sáu bảng được nhóm theo cụm chức năng. Cột "Ràng buộc chính" chỉ nêu các ràng buộc quyết định hành vi của hệ thống; các cột thời gian và khoá ngoại đơn giản được lược bớt cho dễ đọc.

Bảng 3.2. Từ điển dữ liệu: nhóm tài khoản

Bảng	Mục đích	Cột chính	Ràng buộc chính
`user`	Tài khoản đăng nhập	`email`, `password_hash`, `full_name`, `phone`, `role`, `status`	`email` duy nhất; `role` là `CUSTOMER` hoặc `ADMIN`; `status` là `ACTIVE` hoặc `DISABLED`
`refresh_token`	Refresh token đã băm	`user_id`, `token_hash`, `expires_at`, `revoked_at`	`token_hash` duy nhất; xoá theo `user` (`CASCADE`)
`address`	Địa chỉ nhận hàng	`user_id`, `recipient_name`, `phone`, `line`, `is_default`	Đúng một mặc định mỗi khách, service bảo đảm trong giao dịch
`cart`	Giỏ hàng của khách	`user_id`	Mỗi khách một giỏ
`cart_item`	Dòng giỏ hàng	`cart_id`, `variant_id`, `quantity`	Duy nhất theo (`cart_id`, `variant_id`)

Bảng 3.3. Từ điển dữ liệu: nhóm danh mục

Bảng	Mục đích	Cột chính	Ràng buộc chính
`category`	Danh mục sản phẩm	`name`, `status`	`name` duy nhất; ẩn bằng `INACTIVE`
`brand`	Thương hiệu	`name`, `status`	`name` duy nhất; ẩn bằng `INACTIVE`
`product`	Mặt hàng trừu tượng	`name`, `description`, `category_id`, `brand_id`, `status`	Khoá ngoại `RESTRICT` tới danh mục và thương hiệu
`product_variant`	Đơn vị bán và đơn vị tồn kho	`product_id`, `sku`, `size`, `color`, `price`, `stock`	`sku` duy nhất; duy nhất theo (`product_id`, `size`, `color`); `price` và `stock` là số nguyên
`product_image`	Ảnh sản phẩm	`product_id`, `url`, `position`	Duy nhất theo (`product_id`, `position`); tối đa 5 ảnh mỗi sản phẩm ở tầng service

Bảng 3.4. Từ điển dữ liệu: nhóm bán hàng

Bảng	Mục đích	Cột chính	Ràng buộc chính

Bảng	Mục đích	Cột chính	Ràng buộc chính
order	Đơn hàng	code, user_id, client_order_id, status, payment_method, paid_at, subtotal, shipping_fee, total	code duy nhất; duy nhất theo (user_id, client_order_id); paid_at là nguồn sự thật của "đã thanh toán"
order_item	Dòng đơn hàng, ảnh chụp tại lúc đặt	order_id, product_id, variant_id, product_name, variant_sku, size, color, unit_price, unit_cost, quantity	Không cập nhật sau khi tạo; unit_cost có thể rỗng nếu biến thể chưa từng nhập
payment	Một lần thử thanh toán	order_id, provider, status, amount, txn_ref, provider_txn_no, bank_code, pay_date, raw_response	txn_ref duy nhất; mỗi lần thử một dòng; status là PENDING, SUCCEEDED hoặc FAILED

Bảng 3.5. Từ điển dữ liệu: nhóm kho

Bảng	Mục đích	Cột chính	Ràng buộc chính
inventory_receipt	Phiếu nhập kho	code, supplier, note, total_cost, created_by	code dạng PN-…; bất biến sau khi lưu
inventory_receipt_item	Dòng phiếu nhập	receipt_id, variant_id, quantity, unit_cost	Lưu lịch sử giá nhập của từng biến thể
stock_movement	Sổ cái tồn kho	variant_id, delta, reason, ref_type, ref_id, unit_cost	Chỉ ghi thêm; mỗi thay đổi tồn có đúng một dòng

Năm enum dưới đây được khai báo ở tầng cơ sở dữ liệu, không phải chuỗi tự do. Việc dùng enum gốc giúp truy vấn báo cáo không phải xử lý giá trị lạ, nhưng cũng có giá: thêm một trạng thái mới cần một migration có kiểm soát thay vì một dòng dữ liệu mới.

Bảng 3.6. Các enum của lược đồ

Enum	Giá trị	Dùng ở
Role	CUSTOMER, ADMIN	user.role
UserStatus	ACTIVE, DISABLED	user.status
CatalogStatus	ACTIVE, INACTIVE	category, brand, product
OrderStatus	PENDING, CONFIRMED, SHIPPING, DELIVERED, CANCELLED	order.status
PaymentMethod	COD, ONLINE	order.payment_method
PaymentProvider	MOCK, VNPAY	payment.provider
PaymentStatus	PENDING, SUCCEEDED, FAILED	payment.status
StockMovementReason	RECEIPT, SALE, CANCEL_RESTOCK	stock_movement.reason
StockMovementRefType	RECEIPT, ORDER	stock_movement.ref_type

3.4. Sổ cái tồn kho

Tồn kho là chỗ một hệ thống bán hàng dễ sai nhất, nên nó được thiết kế theo nguyên tắc kế toán kép thu nhỏ: cột `product_variant.stock` là số dư hiện tại, và mỗi lần số dư thay đổi đều sinh một dòng `stock_movement` ghi lại `delta` (âm khi bán, dương khi nhập hoặc hoàn), lý do và chứng từ sinh ra nó. Hai bất biến rút ra từ cách làm này, và cả hai đều được kiểm thử:

- Tổng `delta` của một biến thể bằng đúng `stock` hiện tại của nó.
- Mỗi dòng `stock_movement` thuộc đúng một chứng từ: một đơn hàng hoặc một phiếu nhập.

Sổ cái chỉ ghi thêm, không sửa và không xoá. Khi cần điều chỉnh, cách duy nhất là tạo một chứng từ mới sinh ra dòng bù. Cách này khiến câu hỏi "vì sao tồn là 12" luôn trả lời được bằng một truy vấn duy nhất trên sổ cái, thay vì phải suy đoán từ nhiều nguồn.

Ba lý do tồn thay đổi tương ứng với ba giá trị của `reason`: `RECEIPT` khi nhập hàng, `SALE` khi bán (lúc tạo đơn), `CANCEL_RESTOCK` khi huỷ đơn. Việc trừ tồn xảy ra **lúc tạo đơn**, không phải lúc xác nhận thanh toán: nếu chờ đến khi thanh toán xong mới trừ, hai khách có thể cùng mua món hàng cuối cùng. Đây là nội dung của ADR-0002.

3.5. Các giao dịch then chốt

Bốn nghiệp vụ dưới đây chạy trong một giao dịch cơ sở dữ liệu. Chúng là phần dễ sinh dữ liệu không nhất quán nhất, nên mỗi bước được nêu rõ thứ tự.

Đặt hàng. Giao dịch đọc các dòng giỏ, khoá các biến thể bằng `SELECT … FOR UPDATE` theo thứ tự `id` tăng dần để tránh deadlock giữa hai lượt đặt hàng cùng chạm một biến thể, kiểm tra lại sản phẩm còn `ACTIVE` và tồn đủ, tính tạm tính từ giá đã khoá, áp quy tắc phí vận chuyển (mức phẳng, miễn khi tạm tính đạt ngưỡng) rồi ghi đơn với hai con số đã chốt cùng các dòng đơn kèm ảnh chụp tên sản phẩm, SKU, kích cỡ, màu, đơn giá và giá vốn; sau đó trừ tồn và ghi sổ cái cho từng dòng, xoá giỏ, và với đơn online thì tạo dòng `payment` ở trạng thái `PENDING` kèm `txn_ref` mới. Toàn bộ nằm trong một giao dịch, nên không có trạng thái trung gian nào quan sát được từ bên ngoài.

Tạo phiếu nhập kho. Ghi phiếu và các dòng, cộng tồn từng biến thể, ghi một dòng sổ cái mỗi dòng phiếu với `reason = RECEIPT` và `unit_cost` của dòng. Phiếu không có endpoint sửa hay xoá.

Huỷ đơn. Khoá đơn bằng `SELECT … FOR UPDATE`, kiểm tra cặp trạng thái và điều kiện `paid_at` rỗng, hoàn tồn và ghi sổ cái `CANCEL_RESTOCK` cho từng dòng, rồi đặt trạng thái `CANCELLED`. Bước khoá đơn là bắt buộc: nếu hai yêu cầu huỷ đến cùng lúc mà không khoá, tồn sẽ được hoàn hai lần.

Xác nhận thanh toán. Tra `payment` theo `txn_ref`; nếu bản ghi đã `SUCCEEDED` thì trả lời thành công mà không chạm vào đơn, vì cổng thanh toán có thể gọi lại nhiều lần. Ngược lại, kiểm tra số tiền, đặt `payment.status = SUCCEEDED`, ghi `pay_date` và chuyển đơn sang `CONFIRMED` nếu đơn còn `PENDING`, đồng thời ghi `paid_at`. Tính idempotent ở đây đến từ việc kiểm tra trạng thái bản ghi thanh toán, không đến từ khoá ngoài.

3.6. Giá vốn và lãi gộp

Giá vốn của một biến thể tại thời điểm bán là **bình quân gia quyền** của các lần nhập trước đó, tính bằng `SUM(quantity × unit_cost) / SUM(quantity)` trên các dòng phiếu nhập của biến thể. Kết quả được làm tròn tới đồng và chép vào

order_item.unit_cost, nên báo cáo lãi gộp của một đơn cũ không đổi khi sau này cửa hàng nhập thêm hàng với giá khác.

Nếu biến thể chưa từng được nhập kho, unit_cost để rỗng thay vì gán 0. Lựa chọn này quan trọng: gán 0 sẽ khiến báo cáo lãi gộp báo lãi 100% cho những mặt hàng chưa có giá nhập, một con số sai theo hướng lạc quan. Báo cáo chi phí vì vậy trả cả hai chỉ số: lãi gộp tính trên các dòng có giá vốn, và số dòng thiếu giá vốn để người đọc biết độ tin cậy của con số.

Công thức lãi gộp của một kỳ là revenue − cogs, trong đó revenue là tổng order.total của các đơn đã thanh toán trong kỳ và cogs là tổng unit_cost × quantity của các dòng thuộc các đơn đó. Cả hai đều đọc theo paid_at, nên một đơn được ghi nhận doanh thu ở kỳ nào thì giá vốn cũng vào đúng kỳ đó.

3.7. Chỉ mục và truy vấn

Các chỉ mục được đặt theo truy vấn thật của ứng dụng, không theo phỏng đoán. Bảng dưới nêu những chỉ mục không hiển nhiên.

Bảng 3.7. Các chỉ mục phục vụ truy vấn thực tế

Bảng	Chỉ mục	Truy vấn dùng nó
product	(status, created_at)	Danh sách sản phẩm đang bán, sắp mới nhất
product_variant	(product_id, price)	Sắp xếp theo giá trong một sản phẩm và lọc khoảng giá
product_variant	(color)	Bộ lọc màu trên toàn danh mục
order	(user_id, created_at)	Đơn của tôi, mới nhất trước
order	(paid_at)	Báo cáo doanh thu theo khoảng ngày
order	(status)	Báo cáo đơn theo trạng thái và hàng đợi xử lý
stock_movement	(variant_id, created_at)	Lịch sử tồn của một biến thể
stock_movement	(ref_type, ref_id)	Truy vết chứng từ sinh ra một thay đổi tồn
refresh_token	(user_id, expires_at)	Thu hồi token hết hạn của một người dùng

Tìm kiếm theo tên dùng LIKE '%…%' trên cột name với chuỗi đã bỏ dấu ở cả hai phía; vì vậy người dùng gõ "ao thun" vẫn tìm được "Áo thun basic". Với quy mô dữ

liệu của đồ án (185 sản phẩm), cách này nhanh hơn nhiều so với dựng một chỉ mục toàn văn, và nó không cần thêm thành phần hạ tầng nào. Nếu danh mục lớn lên vài chục nghìn dòng, bước tiếp theo là một cột tên đã chuẩn hoá kèm chỉ mục, hoặc chuyển sang `FULLTEXT` với bộ tách từ tiếng Việt.

3.8. Bốn quyết định miền

ADR-0001 — thanh toán theo từng lần thử. Một đơn có thể có nhiều bản ghi `payment`, mỗi lần thử một bản ghi với `txn_ref` riêng. Lý do: cổng thanh toán thất bại thường do phía ngân hàng, và người dùng cần thử lại mà không phải tạo đơn mới. Nếu mỗi đơn chỉ một bản ghi thanh toán thì lần thử thứ hai sẽ ghi đè lịch sử của lần thứ nhất, làm mất dấu vết khi đối soát.

ADR-0002 — sổ cái tồn kho và trừ tồn lúc đặt hàng. Tồn là một cột, nhưng mọi thay đổi đều có dòng sổ cái trong cùng giao dịch; tồn bị trừ ngay khi đơn được tạo. Lý do: trừ muộn cho phép bán quá tồn, còn chỉ dùng sổ cái mà không có cột số dư thì mỗi lần đọc tồn phải cộng toàn bộ lịch sử.

ADR-0003 — tiền là số nguyên VND. Mọi cột tiền là `INT`, đơn vị đồng, không có phần lẻ. Lý do: VND không dùng phần thập phân trong thực tế, và số nguyên loại bỏ hoàn toàn sai số làm tròn khi cộng doanh thu. Đổi lại, hệ thống không biểu diễn được giá có phần lẻ; nếu cần, đó là một thay đổi lược đồ có chủ đích chứ không phải một tuỳ chọn của từng bảng.

ADR-0004 — đơn hàng lưu ảnh chụp. Dòng đơn hàng chép lại tên sản phẩm, SKU, kích cỡ, màu, đơn giá và giá vốn tại thời điểm đặt, đồng thời chép cả người nhận, số điện thoại và địa chỉ giao. Lý do: giá đổi, sản phẩm đổi tên, địa chỉ bị sửa — nhưng hoá đơn của khách thì không được đổi theo. Vì vậy `order` không có khoá ngoại tới `address`, và `order_item` giữ `product_name` dù vẫn trỏ tới `product_id` để truy vết.

4. THIẾT KẾ GIAO DIỆN

4.1. Định hướng thiết kế

Ứng dụng bán quần áo có một nghịch lý: người dùng đến để mua hàng, nhưng thứ họ nhìn nhiều nhất lại là ảnh sản phẩm và giá. Giao diện vì vậy phải nhường chỗ cho ảnh, và mọi thành phần khác chỉ được phép xuất hiện khi nó trả lời một câu hỏi của người mua. Ba nguyên tắc dưới đây dẫn dắt toàn bộ chương này.

Ấm và tĩnh. Bảng màu là ivory, mực và terracotta; chữ tiêu đề dùng serif, phần thân dùng sans. Lựa chọn này khác với mặc định xanh–tím của Material 3, vì một cửa hàng quần áo cần cảm giác giấy và vải chứ không phải cảm giác phần mềm doanh nghiệp. Chỉ có một theme sáng: không dark mode, không chuyển theme động, vì mỗi theme thêm vào là một bộ ảnh và một bộ kiểm thử độ tương phản phải làm lại.

Phẳng, phân tách bằng bề mặt. Thẻ sản phẩm không có đổ bóng, không có viền. Việc phân tách dựa vào chênh lệch độ sáng giữa thẻ và nền, và chênh lệch đó được đo chứ không được cảm nhận: thẻ trắng `#FFFFFF` trên nền `#F6F1E8` đạt tỉ lệ tương phản 1.13:1, đủ nhận ra ranh giới; màu `#FBF8F1` chỉ đạt 1.06:1 nên không được dùng làm thẻ.

Thuật ngữ thuần Việt. Chuỗi hiển thị lấy từ bảng thuật ngữ của dự án: "Kích cỡ" chứ không phải "Size", "Biến thể" trong ngữ cảnh quản trị, "Thêm S vào giỏ" thay cho "Thêm vào giỏ" để người dùng biết mình đang mua kích cỡ nào. Tên model trong code (`Variant`, `ProductVariant`) không xuất hiện trên giao diện.

4.2. Hệ thống thiết kế

Toàn bộ màu, chữ và hình dạng đi qua một theme duy nhất (`StyloTheme`). Không màn hình nào được đặt mã màu trực tiếp; đổi một token là đổi cả ứng dụng.

4.2.1. Màu

Bảng dưới là các vai trò màu của Compose `ColorScheme` cùng giá trị hex. Cột "Dùng cho" chỉ ra nơi vai trò đó thực sự xuất hiện, để tránh việc khai báo đủ vai trò rồi chỉ dùng ba màu.

Bảng 4.1. Bảng màu của hệ thống thiết kế

Vai trò	Hex	Dùng cho
primary / onPrimary	#1E1C18 / #FBF8F1	Nút chính, chip đang chọn, tiêu đề
secondary / onSecondary	#A6512B / #FFF6EF	Nhấn mạnh thương hiệu, liên kết, giá khuyến mãi
tertiary / onTertiary	#4F5E3F / #FFFFFF	Trạng thái thành công, nhãn "Đang bán"
error / onError	#B3261E / #FFFFFF	Lỗi, hành động xoá, huỷ đơn
background / onBackground	#F6F1E8 / #1E1C18	Nền toàn ứng dụng
surface / onSurface	#FBF8F1 / #1E1C18	Thanh trên, thanh dưới, sheet
surfaceContainerLowest	#FFFFFF	Thẻ sản phẩm, thẻ KPI, thẻ đơn
surfaceVariant / onSurfaceVariant	#EDE5D8 / #6C6255	Chip chưa chọn, nhãn phụ
surfaceDim	#E9E3D8	Khung xương khi tải
outline / outlineVariant	#8F8069 / #E8DFD1	Viền control, đường phân cách

Năm màu trạng thái (success, warning, info, indigo, neutral) không có trong ColorScheme của Material 3, nên chúng được cung cấp qua một đối tượng riêng (StyloStatusColors) với cặp container/onContainer. Nhờ vậy nhãn trạng thái đơn hàng có cùng một bộ màu ở mọi màn hình thay vì mỗi nơi tự chọn.

4.2.2. Chữ

Hai họ chữ, không nhúng tệp font: serif hệ thống cho tiêu đề, sans hệ thống cho phần còn lại. Bảng dưới là các token đang dùng; không màn hình nào đặt cỡ chữ trực tiếp.

Bảng 4.2. Thang chữ của hệ thống thiết kế

Vai trò	Token Compose	Cỡ / dòng	Kiểu
Hero trang chủ	headlineMedium	28sp / 36sp	Serif, semibold
Tiêu đề màn hình, KPI	headlineSmall	24sp / 30sp	Serif, semibold
Tiêu đề thẻ	titleLarge	18sp / 24sp	Serif, semibold
Giá lớn	titleMedium	16sp / 24sp	Sans, semibold
Giá thường	titleSmall	14sp / 20sp	Sans, semibold
Thân bài	bodyMedium	14sp / 20sp	Sans
Nút và chip	labelLarge	14sp / 20sp	Sans, medium
Nhãn, metadata	labelMedium	12sp / 16sp	Sans, medium

Vai trò	Token Compose	Cỡ / dòng	Kiểu
Overline	`labelSmall`	11sp / 16sp	Sans, tracking 1.5sp

Số tiền luôn dùng `Formatters.money` và nối ký hiệu ₫ bằng khoảng trắng không ngắt (NBSP), nên giá không bao giờ bị tách dòng giữa số và đơn vị. Trên màn hình khách, số tiền viết đầy đủ; riêng thẻ KPI của quản trị dùng "tr" cho gọn, và quy ước này được ghi trong tài liệu thiết kế để không ai phải đoán.

4.2.3. Khoảng cách, hình dạng, độ nổi

Lưới cơ sở là 4dp; lề ngang của trang là 20dp; khoảng cách giữa các section là 24dp, giữa các thẻ trong lưới là 14dp, giữa các dòng trong một danh sách là 10–12dp. Thang bo góc: 4, 8, 12, 18, 24. Thẻ dùng 12, ảnh trong thẻ dùng 12, ảnh thu nhỏ dùng 8, ô nhập và menu dùng 8, chip và nút dùng bo tròn hoàn toàn, hộp thoại và sheet dùng 24 (sheet chỉ bo hai góc trên).

Không dùng đổ bóng ở bất kỳ đâu. Phân cấp thị giác đến từ ba thứ: bề mặt (thẻ trắng trên nền ivory), khoảng trắng, và trọng lượng chữ. Cách này giữ được giao diện nhẹ và tránh hiệu ứng "mọi thứ đều nổi" thường thấy khi lạm dụng elevation.

4.2.4. Bộ thành phần dùng chung

Một phát hiện từ đợt đánh giá giao diện (mục 4.7) là các điều khiển giống nhau bị dựng lại ở nhiều màn với ba bốn biến thể khác nhau. Sau khi sửa, mọi điều khiển lặp lại chỉ tồn tại ở một nơi, trong gói `core/ui/components`; màn hình chỉ còn bố cục và trạng thái.

Bảng 4.3. Các thành phần dùng chung và nơi chúng xuất hiện

Thành phần	Vai trò	Xuất hiện ở
`StyloCard`	Bề mặt thẻ phẳng, bo 12, không viền	Trang chủ, danh sách, quản trị
`StyloSearchField`	Ô tìm kiếm dạng pill cao 48dp	Trang chủ, tìm kiếm, quản trị
`StyloFilterChip`	Chip lọc có trạng thái chọn kèm icon	Tìm kiếm, bộ lọc, quản trị
`StyloSwatch`	Ô màu tròn kèm tên màu	Chi tiết sản phẩm
`StyloSizeChip`	Chip kích cỡ	Chi tiết sản phẩm
`StyloQtyStepper`	Bộ tăng giảm số lượng, tự chặn ở tồn	Chi tiết, giỏ hàng, phiếu nhập

Thành phần	Vai trò	Xuất hiện ở
`StyloBusyButton`	Nút có trạng thái đang xử lý	Đăng nhập, đặt hàng, lưu
`StyloStickyCta`	Thanh dưới cố định: giá bên trái, nút bên phải	Chi tiết, giỏ, thanh toán
`StyloStatusBadge`	Nhãn trạng thái theo bộ màu chung	Đơn hàng, sản phẩm, người dùng
`StyloListSkeleton`	Khung xương khi tải	Mọi danh sách
`StyloEmptyState`	Trạng thái rỗng kèm hành động	Giỏ, đơn, danh sách quản trị
`StyloErrorState`	Trạng thái lỗi kèm nút thử lại	Mọi màn tải dữ liệu
`StyloInlineError`	Lỗi ngay dưới ô nhập	Biểu mẫu
`StyloConfirmDialog`	Hộp thoại xác nhận hành động không hoàn tác	Huỷ đơn, đăng xuất, xoá

4.2.5. Trạng thái chung

Bốn trạng thái dữ liệu được thiết kế một lần và dùng lại: **đang tải** là khung xương có đúng số thẻ của lưới, không phải vòng xoay giữa màn hình; **rỗng** là một câu giải thích cùng một hành động ("Chưa có đơn hàng" kèm nút về trang chủ); **lỗi** là thông điệp tiếng Việt đọc từ mã lỗi của API kèm nút "Thử lại"; **lỗi cục bộ** nằm ngay dưới ô nhập và không đẩy cả biểu mẫu đi. Việc ánh xạ mã lỗi sang câu tiếng Việt nằm ở một chỗ duy nhất (`ErrorMessages`), nên API đổi mã lỗi sẽ lộ ra ngay chứ không âm thầm hiển thị câu cũ.

4.3. Điều hướng theo vai trò

Phía khách có năm tab dưới: Trang chủ, Tìm kiếm, Giỏ hàng, Đơn hàng, Tôi. Năm tab này phản ánh đúng năm việc người mua làm, và giỏ hàng có huy hiệu số lượng để trả lời câu hỏi "tôi đã thêm gì rồi".



Hình 4.1. Trang chủ: hero, chip danh mục, dải miễn phí giao hàng và hai khối sản phẩm

Phía quản trị giữ bốn tab vận hành thường ngày (Tổng quan, Đơn hàng, Sản phẩm, Kho) và đưa các mục ít dùng hơn (Danh mục & thương hiệu, Người dùng, Báo cáo, Đăng xuất) vào một ngăn kéo mở từ góc trên. Lý do: người quản trị mở ứng dụng hàng chục lần mỗi ngày để xử lý đơn và kiểm tra tồn, nhưng rất ít khi mở báo cáo; đặt báo cáo ngang hàng với đơn hàng sẽ làm chậm việc thường ngày.

Điều hướng được mount theo vai trò: đồ thị quản trị chỉ tồn tại khi tài khoản là `ADMIN`, và khi đổi tab quản trị, trạng thái cuộn cùng bộ lọc của tab cũ được lưu lại. Người dùng không thấy màn hình quản trị bị chớp qua khi đăng nhập bằng tài khoản khách.

4.4. Màn hình phía khách

4.4.1. Đăng nhập và đăng ký

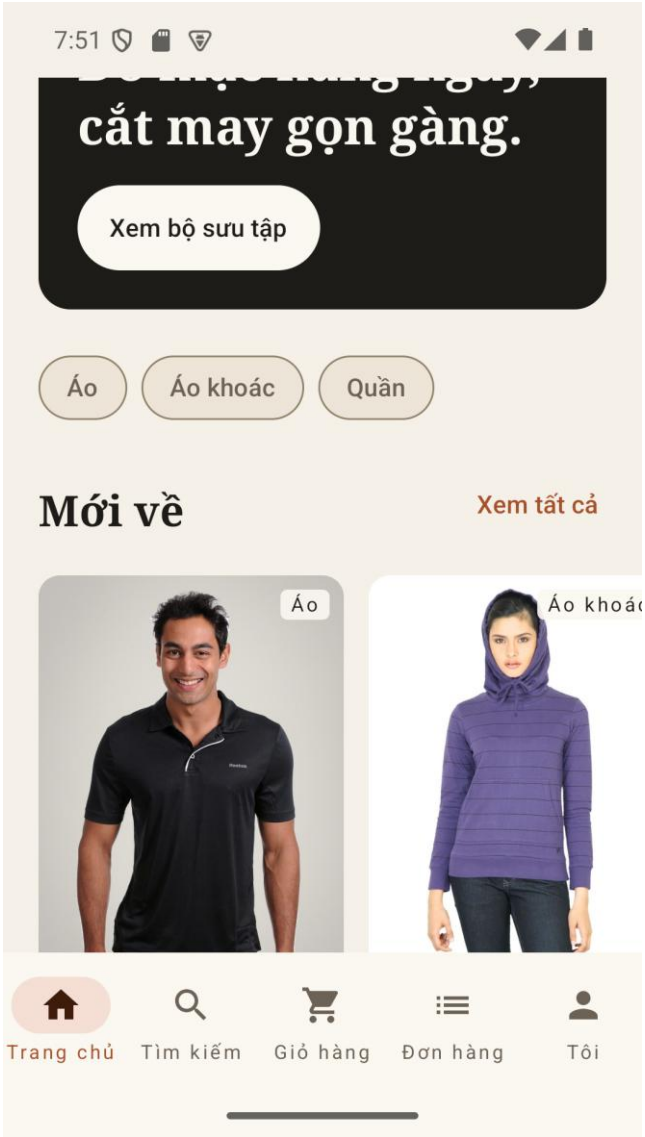
Một màn hình hai chế độ, chuyển bằng segmented control. Đăng nhập chỉ hỏi email và mật khẩu; đăng ký hỏi thêm họ tên và số điện thoại (không bắt buộc). Mọi lỗi đăng nhập đều trả về cùng một câu "Email hoặc mật khẩu không đúng" để không tiết lộ email nào đã tồn tại; lỗi đăng ký thì chỉ rõ từng ô. Nút gửi có trạng thái đang xử lý, và bàn phím không che nút nhờ cuộn được.



Hình 4.2. Biểu mẫu đăng ký với bốn ô và nút tạo tài khoản

4.4.2. Trang chủ

Bố cục A: hero toàn chiều rộng, dải chip danh mục, carousel "Mới về" tám sản phẩm, dải miễn phí giao hàng, rồi hai khối danh mục "Áo" và "Quần", mỗi khối bốn sản phẩm kèm liên kết "Xem tất cả". Trang chủ không cần đăng nhập và không có section "bán chạy": API không cung cấp sắp xếp theo số bán, nên dựng một section như vậy sẽ phải bịa dữ liệu.

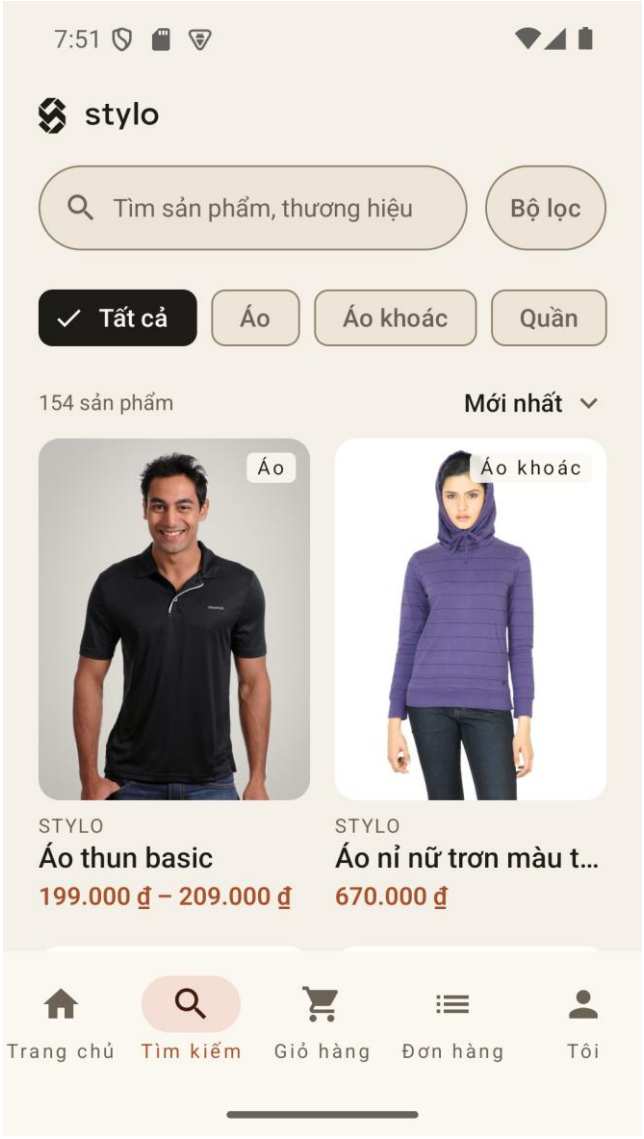


Hình 4.3. Trang chủ sau khi cuộn: carousel "Mới về" và hai khối danh mục

4.4.3. Tìm kiếm và lọc

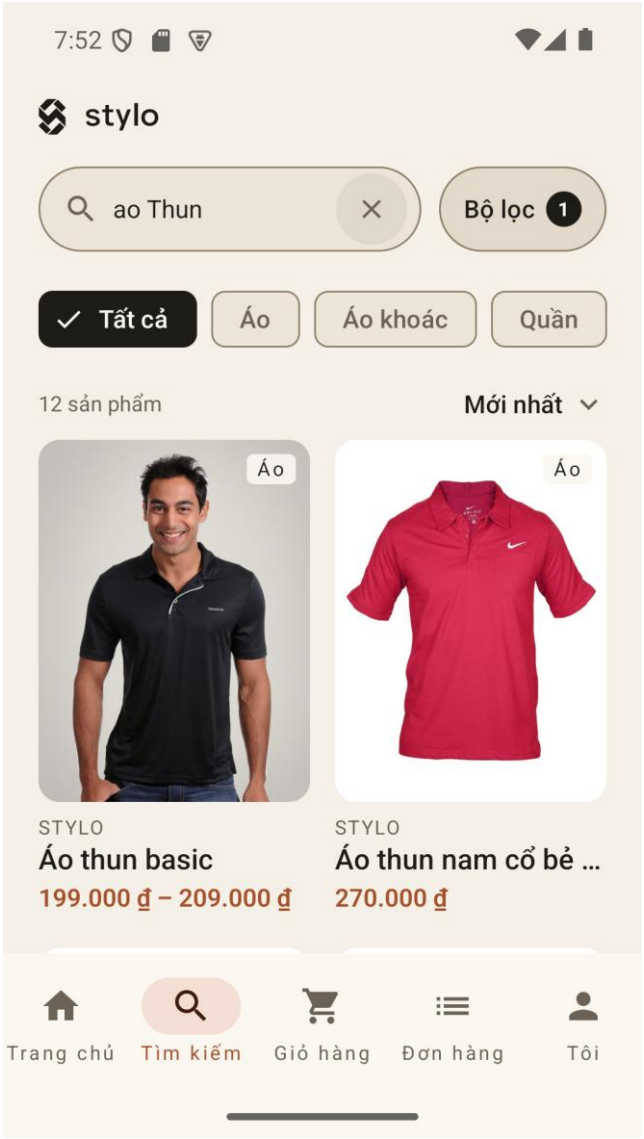
Màn hình tìm kiếm là một ô tìm kiếm dính trên, dải chip danh mục, nút "Bộ lọc", bộ chọn sắp xếp, dòng đếm kết quả và lưới hai cột. Bộ lọc mở trong một sheet từ dưới

lên, gồm danh mục, thương hiệu, màu, kích cỡ và khoảng giá; nút "Áp dụng" nằm cuối sheet và chỉ đóng sheet khi bộ lọc đã được gửi đi.



Hình 4.4. Màn hình tìm kiếm với chip danh mục, sắp xếp và lưới hai cột

Tìm kiếm không phân biệt dấu: gõ "ao thun" khớp "Áo thun basic". Bộ lọc kết hợp được với nhau và với từ khoá, vì tất cả đi trong một yêu cầu duy nhất tới `GET /products`.



Hình 4.5. Kết quả cho từ khoá "ao thun": 39 sản phẩm, không cần gõ dấu



Hình 4.6. Bộ lọc dạng sheet: danh mục, thương hiệu, màu sắc

4.4.4. Chi tiết sản phẩm

Trang chi tiết là nơi giao diện phải trả lời ba câu hỏi liên tiếp: món này trông thế nào, còn kích cỡ nào, và giá của đúng biến thể đang chọn là bao nhiêu. Vì vậy thứ tự các khối là ảnh, thông tin và giá, chọn màu, chọn kích cỡ, tồn kho của biến thể, số lượng, rồi mô tả. Giá và tồn cập nhật ngay khi đổi biến thể; khi tồn xuống thấp, nhãn chuyển sang màu cảnh báo kèm chữ "(tồn thấp)".

Hai chi tiết nhỏ nhưng quyết định trải nghiệm mua. Thứ nhất, app chọn sẵn **màu đầu tiên còn hàng** chứ không phải màu đầu tiên trong danh sách, nên một sản phẩm còn bán không bao giờ mở ra ở trạng thái "hết hàng" chỉ vì màu xếp trước đã hết. Thứ hai, bộ tăng giảm số lượng nằm ngay trên thanh mua, giới hạn 1 đến tồn của đúng biến

thể đang chọn; đổi màu hoặc kích cỡ thì số lượng được kẹp lại theo tồn mới và thêm thành công thì trở về 1. Server vẫn là nơi quyết định: nó cộng dồn vào dòng đã có trong giỏ và trả `OUT_OF_STOCK` nếu tổng vượt tồn, nên giao diện chỉ ngăn trước chứ không thay thế kiểm tra phía server.



Hình 4.7. Chi tiết sản phẩm: ảnh, thương hiệu, tên và giá

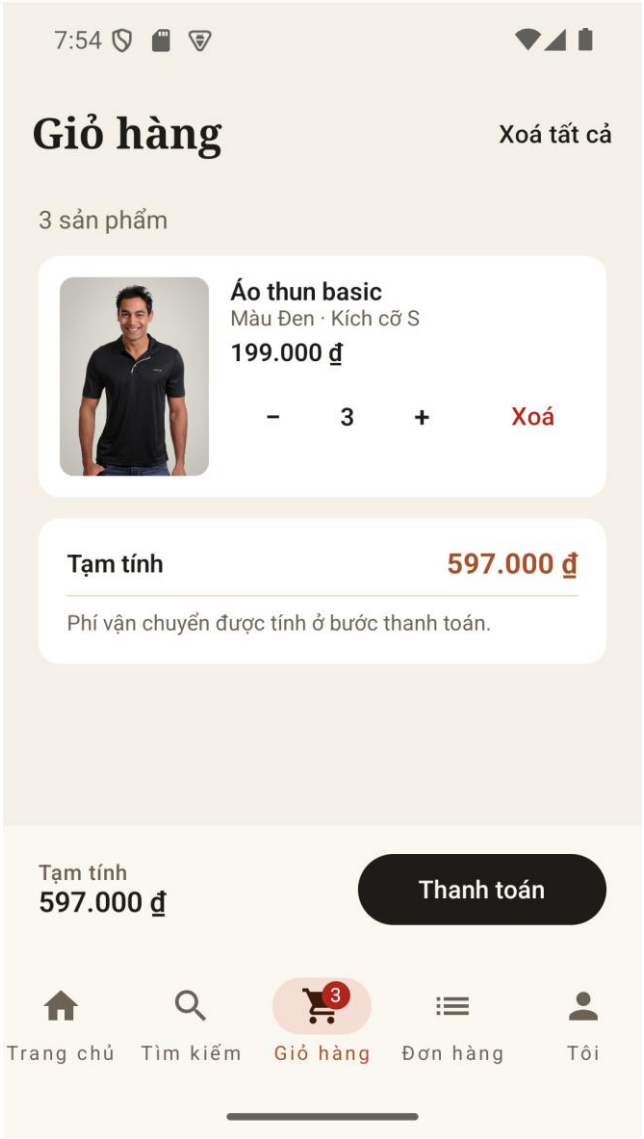


Hình 4.8. Chọn màu và kích cỡ; tồn kho và số lượng hiển thị theo biến thể đang chọn

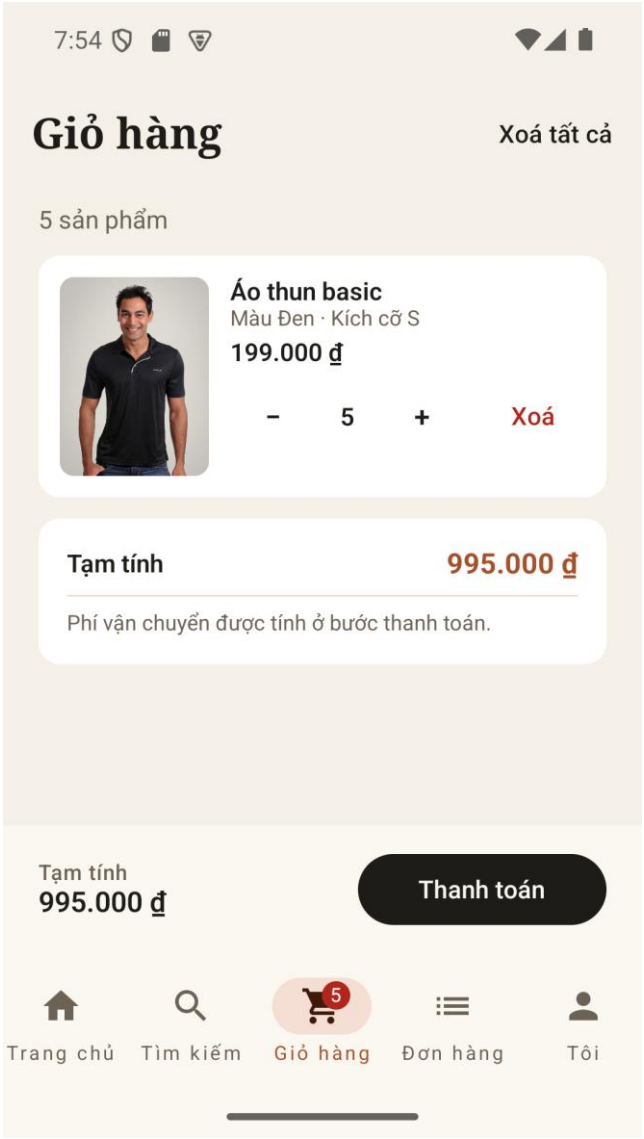
Nút mua ở thanh dưới luôn nêu kích cỡ đang chọn ("Thêm M vào giỏ"), một quyết định đến từ đợt đánh giá giao diện: nút chỉ ghi "Thêm vào giỏ" khiến người dùng không chắc mình đã chọn đúng cỡ hay chưa.

4.4.5. Giỏ hàng

Mỗi dòng giỏ hiển thị ảnh, tên, tổ hợp màu và kích cỡ, đơn giá, bộ tăng giảm số lượng và nút xoá. Bộ tăng giảm tự chặn ở số lượng tồn và hiện nhãn "Tối đa" khi chạm trần, nên người dùng biết vì sao không tăng được nữa thay vì thấy nút im lặng. Thanh dưới hiển thị tạm tính và nút thanh toán; phí vận chuyển được nêu rõ là tính ở bước sau.



Hình 4.9. Giỏ hàng với một dòng, bộ tăng giảm và tạm tính



Hình 4.10. Tăng số lượng ngay trong giỏ; tạm tính và huy hiệu giỏ cập nhật theo

4.4.6. Thanh toán và kết quả

Màn hình thanh toán gồm địa chỉ nhận (kèm nút "Đổi"), danh sách sản phẩm rút gọn, hai phương thức thanh toán, ô ghi chú không bắt buộc và thanh dưới chứa tổng tiền. Nút đặt hàng ghi cả số tiền để người dùng không phải nhớ.

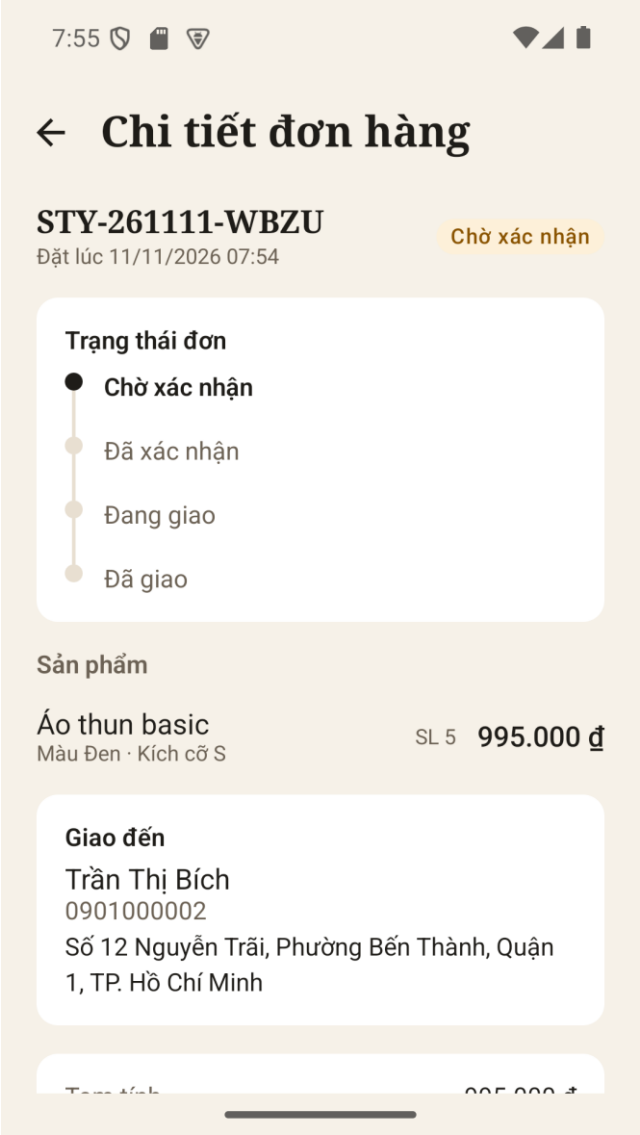
Thanh dưới tách ba dòng: tạm tính, phí vận chuyển và tổng cộng. Cả ba lấy từ `GET /cart`, trong đó `shippingFee` là bản xem trước của đúng quy tắc server dùng lúc đặt hàng: một mức phẳng 30.000 ₫, miễn khi tạm tính đạt 500.000 ₫. Phí bằng 0 hiển thị "Miễn phí" thay vì "0 ₫", vì người mua đọc ưu đãi dễ hơn đọc số không; nút đặt hàng dùng tổng cộng chứ không dùng tạm tính. Trước đây màn hình chỉ hiện tạm tính

kèm ghi chú "phí xác nhận khi đặt hàng", và đó là chỗ dễ gây bất ngờ nhất khi tổng tiền tăng sau khi bấm.



Hình 4.11. Thanh toán: tạm tính, phí vận chuyển (miễn phí khi đạt ngưỡng) và tổng cộng

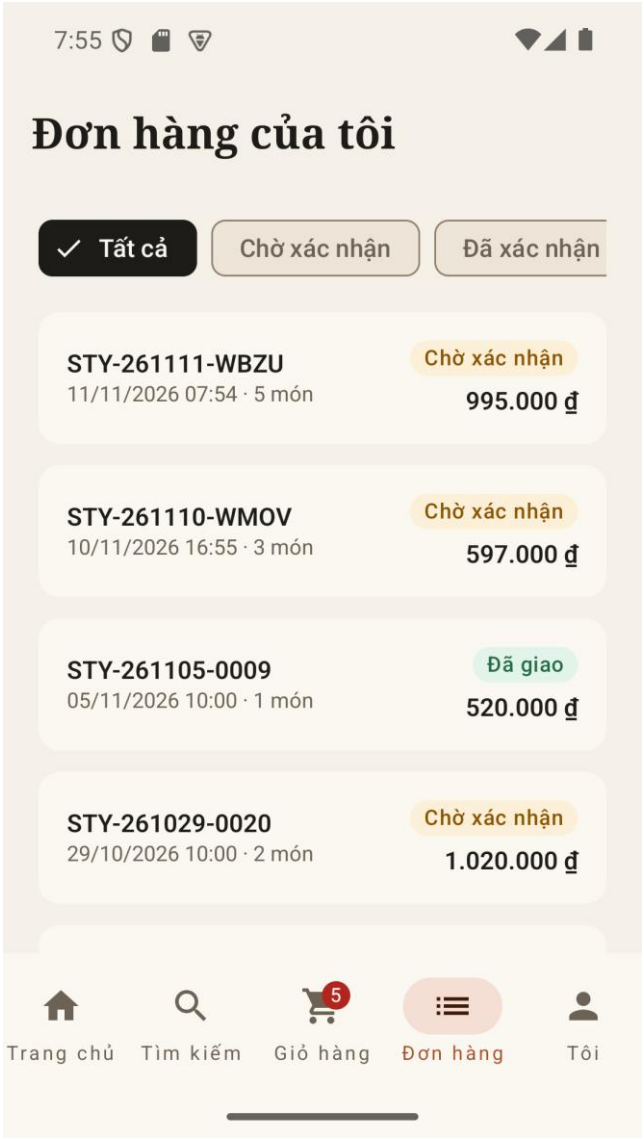
Với COD, đơn được tạo ngay và mở trang chi tiết đơn ở trạng thái "Chờ xác nhận". Với thanh toán online, ứng dụng mở cổng thanh toán trong trình duyệt; khi quay lại, trạng thái đơn được đọc lại từ server chứ không tin vào tham số trên đường dẫn.



Hình 4.12. Chi tiết đơn vừa đặt, có thanh trạng thái bốn mốc

4.4.7. Đơn hàng

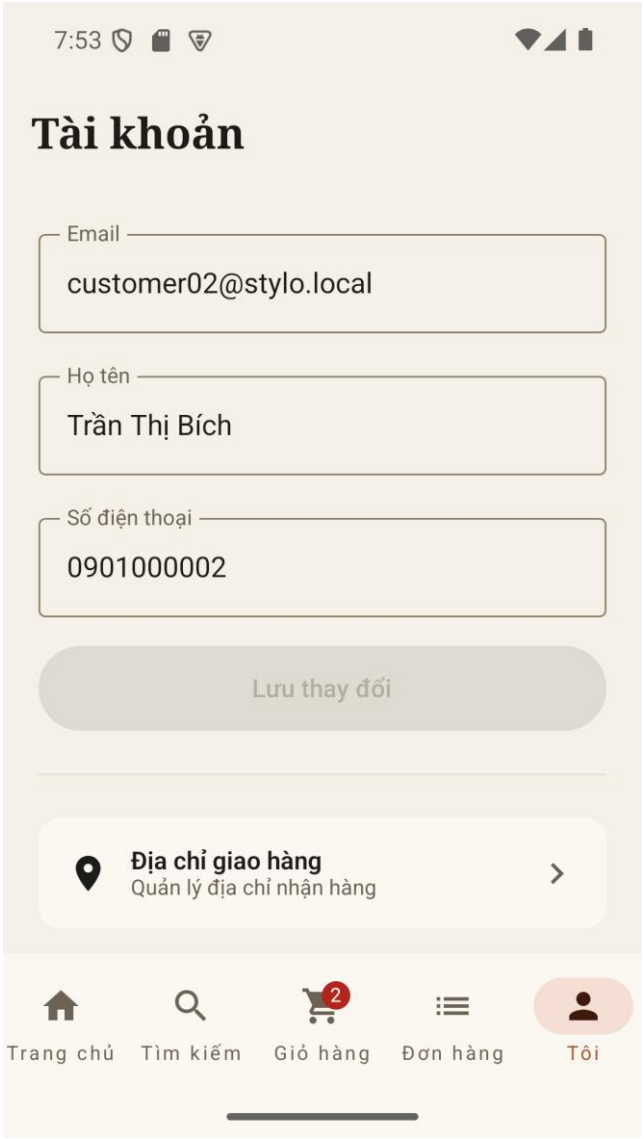
Danh sách đơn chia theo bốn tab trạng thái, mỗi thẻ nêu mã đơn, thời điểm, số món, trạng thái và tổng tiền. Chi tiết đơn hiển thị thanh tiến trình bốn mốc, sản phẩm, địa chỉ giao và tạm tính; hành động huỷ chỉ xuất hiện khi đơn còn huỷ được.



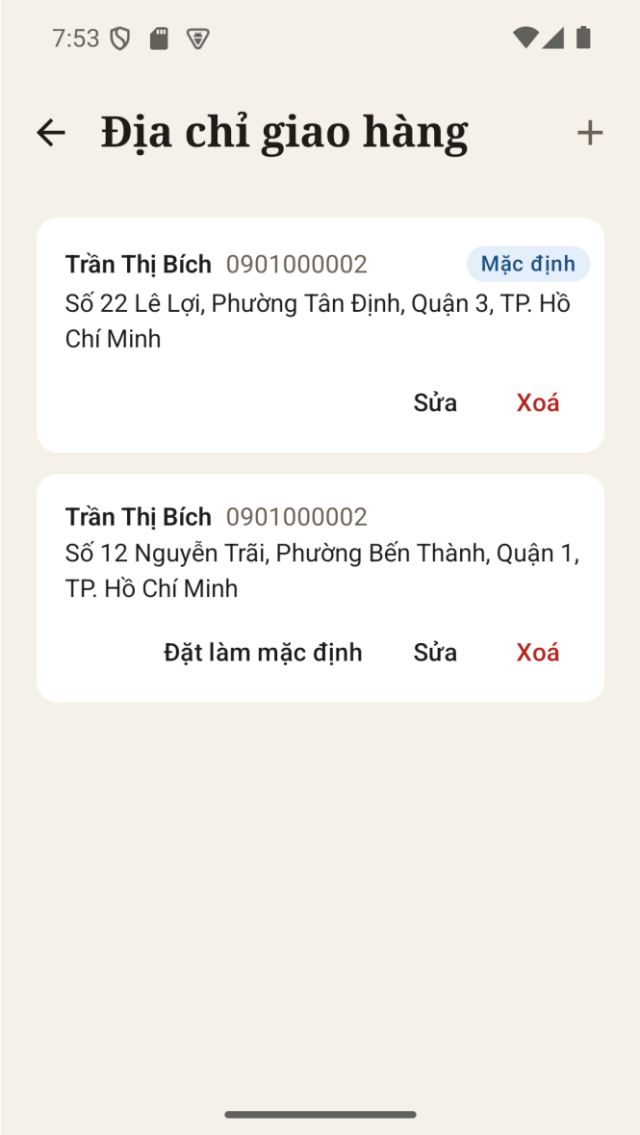
Hình 4.13. Danh sách đơn hàng của tôi theo tab trạng thái

4.4.8. Hồ sơ và địa chỉ

Hồ sơ cho sửa họ tên và số điện thoại, email chỉ đọc. Danh sách địa chỉ đánh dấu rõ địa chỉ mặc định, mỗi dòng có "Sửa", "Xoá" và "Đặt làm mặc định"; biểu mẫu thêm địa chỉ gồm người nhận, số điện thoại và một dòng địa chỉ tự do.



Hình 4.14. Hồ sơ khách hàng: thông tin, địa chỉ, đăng xuất



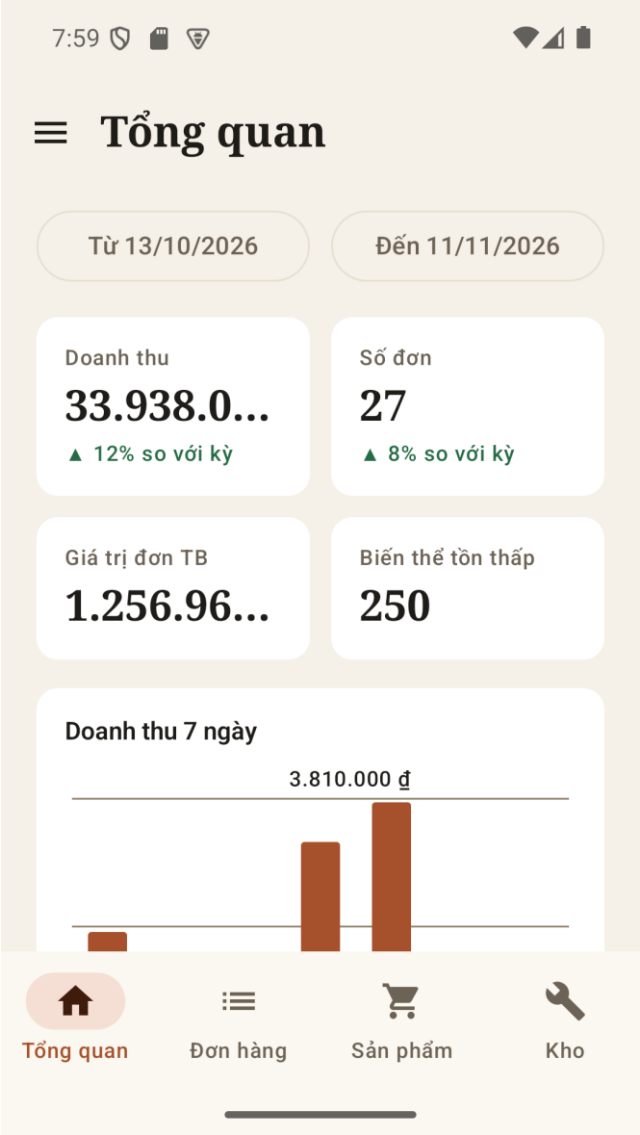
Hình 4.15. Danh sách địa chỉ với địa chỉ mặc định được đánh dấu

Một dòng địa chỉ tự do thay vì ba ô chọn tỉnh/quận/phường là lựa chọn có chủ đích: bộ dữ liệu hành chính ba cấp cần dữ liệu nguồn và luôn lệch sau mỗi lần sáp nhập đơn vị hành chính, trong khi đồ án chỉ cần một địa chỉ đủ để giao hàng.

4.5. Màn hình phía quản trị

4.5.1. Tổng quan

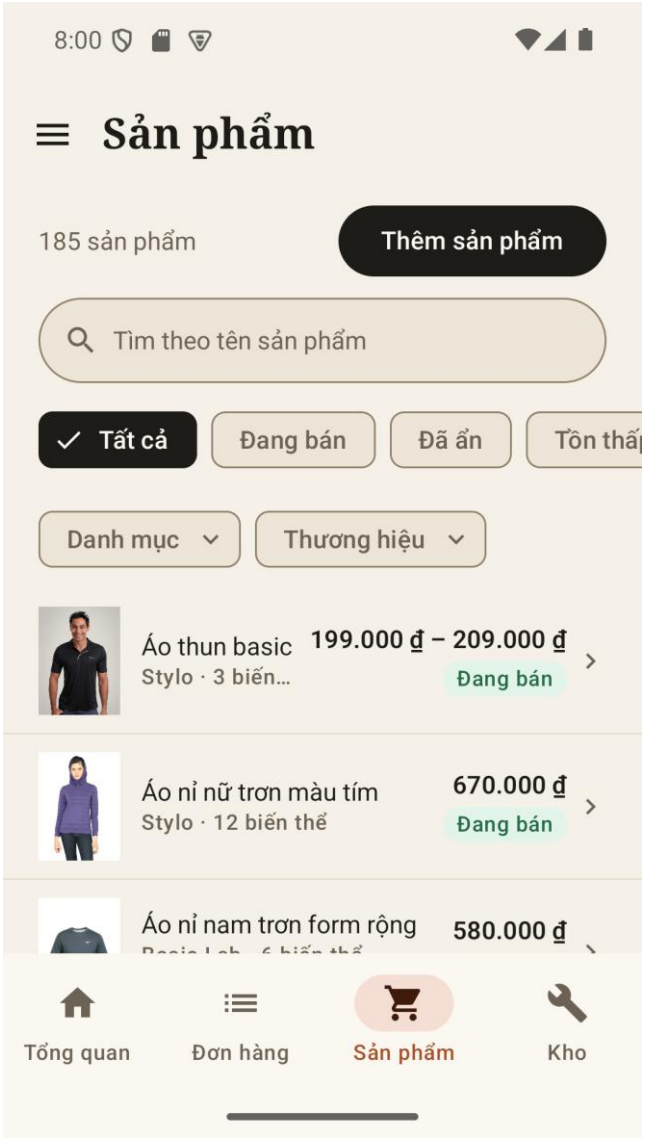
Bốn thẻ KPI (doanh thu, số đơn, giá trị đơn trung bình, số biến thể tồn thấp), biểu đồ cột doanh thu bảy ngày, danh sách sản phẩm bán chạy và đơn gần đây. Biểu đồ có nhãn đọc cho trình đọc màn hình, liệt kê từng ngày kèm doanh thu và số đơn, nên thông tin không chỉ tồn tại dưới dạng hình.



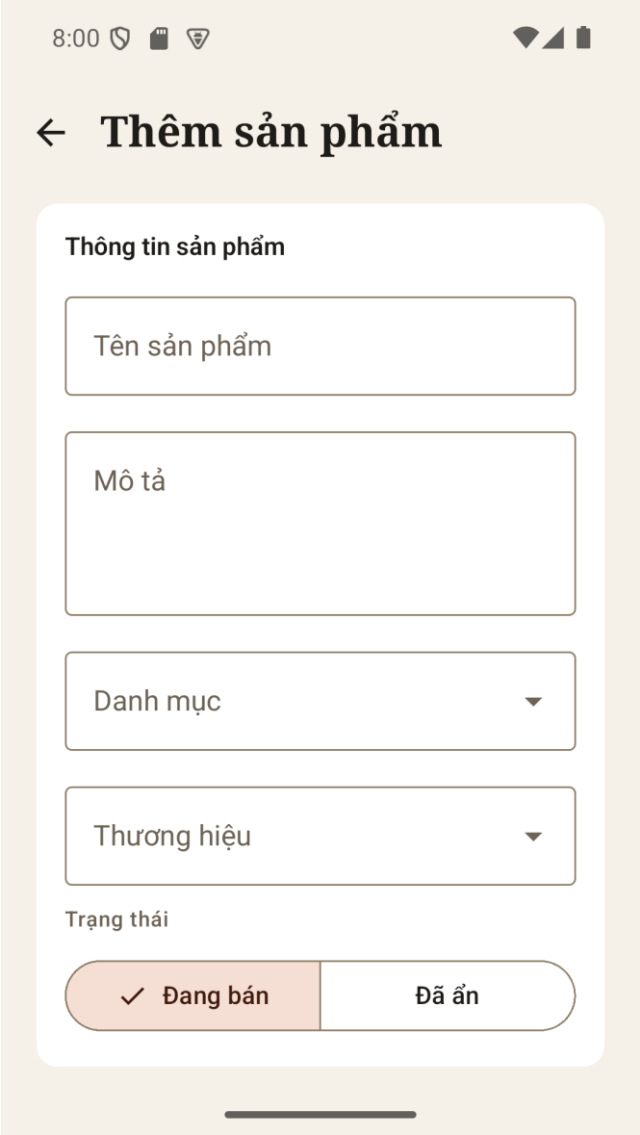
Hình 4.16. Tổng quan quản trị: bốn KPI và biểu đồ doanh thu bảy ngày

4.5.2. Sản phẩm

Danh sách sản phẩm có tìm kiếm, bộ lọc trạng thái (Tất cả, Đang bán, Đã ẩn, Tồn thấp) và bộ lọc danh mục, thương hiệu. Mỗi dòng nêu số biến thể và khoảng giá. Biểu mẫu sản phẩm gồm tên, mô tả, danh mục, thương hiệu và trạng thái; biến thể và ảnh được quản lý ở màn hình riêng của sản phẩm.



Hình 4.17. Danh sách sản phẩm quản trị với trạng thái và số biến thể



Hình 4.18. Biểu mẫu thêm sản phẩm

4.5.3. Đơn hàng

Hàng đợi đơn chia tab theo trạng thái, lọc nhanh theo thời gian (hôm nay, 7 ngày, 30 ngày). Chi tiết đơn của quản trị nhiều hơn của khách hai phần: giá vốn từng dòng, và khối "Xử lý đơn" chứa các hành động hợp lệ với trạng thái hiện tại. Mỗi hành động mở hộp thoại xác nhận nêu rõ cặp trạng thái sẽ chuyển, ví dụ "Chờ xác nhận → Đã xác nhận", để thao tác không thể nhầm.



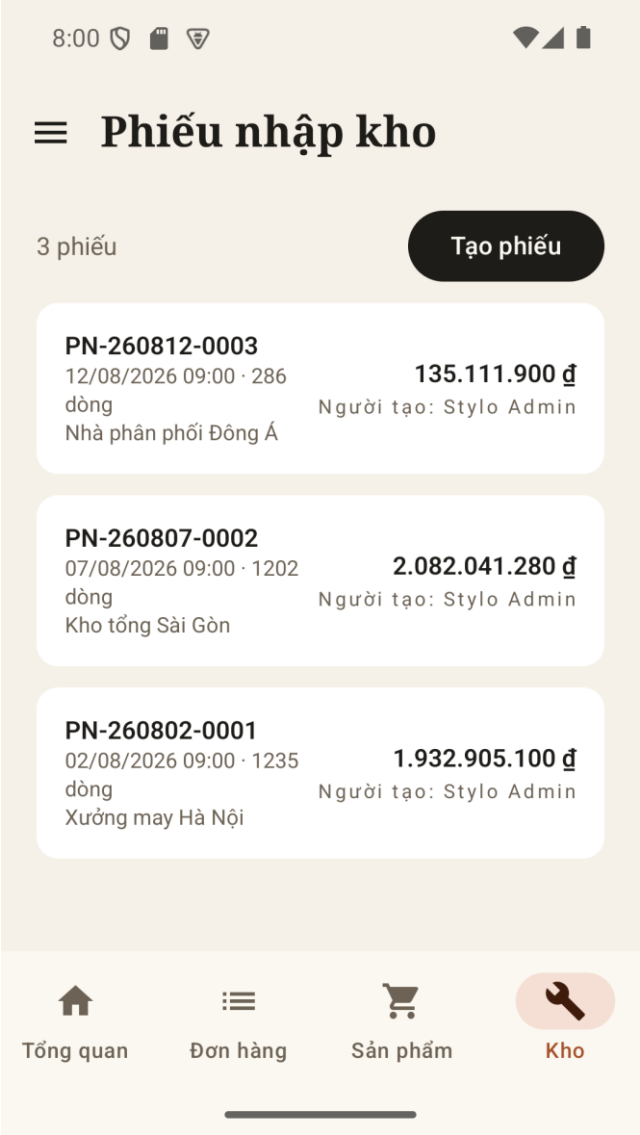
Hình 4.19. Hàng đợi đơn hàng của quản trị



Hình 4.20. Hộp thoại xác nhận chuyển trạng thái

4.5.4. Kho và phiếu nhập

Màn hình kho liệt kê phiếu nhập và tồn thấp. Phiếu nhập nhiều dòng: mỗi dòng chọn biến thể, số lượng và đơn giá nhập; tổng tiền phiếu cộng dần khi thêm dòng. Sau khi lưu, phiếu không sửa được nữa, đúng như quy tắc nghiệp vụ ở chương 2.



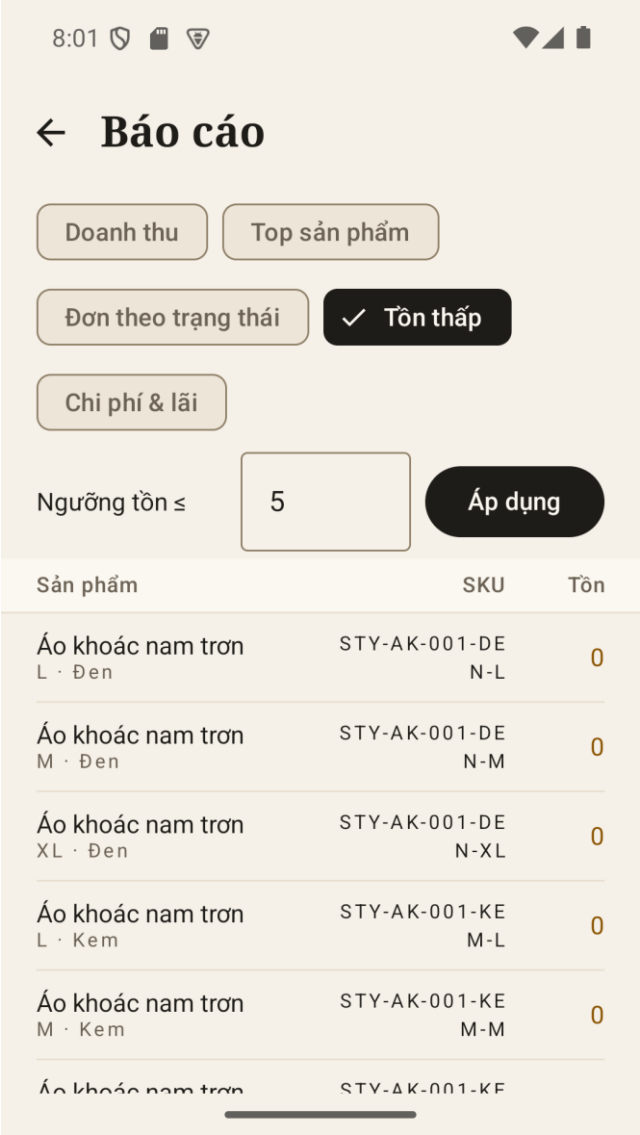
Hình 4.21. Kho: danh sách phiếu nhập và biến thể tồn thấp

4.5.5. Báo cáo

Năm báo cáo nằm trong một màn hình, chuyển bằng segmented control, dùng chung một bộ chọn khoảng ngày. Báo cáo doanh thu có thêm lựa chọn nhóm theo ngày hoặc theo tháng. Biểu đồ luôn kèm bảng số liệu bên dưới, nên người đọc có thể lấy số chính xác mà không phải ước lượng từ cột.



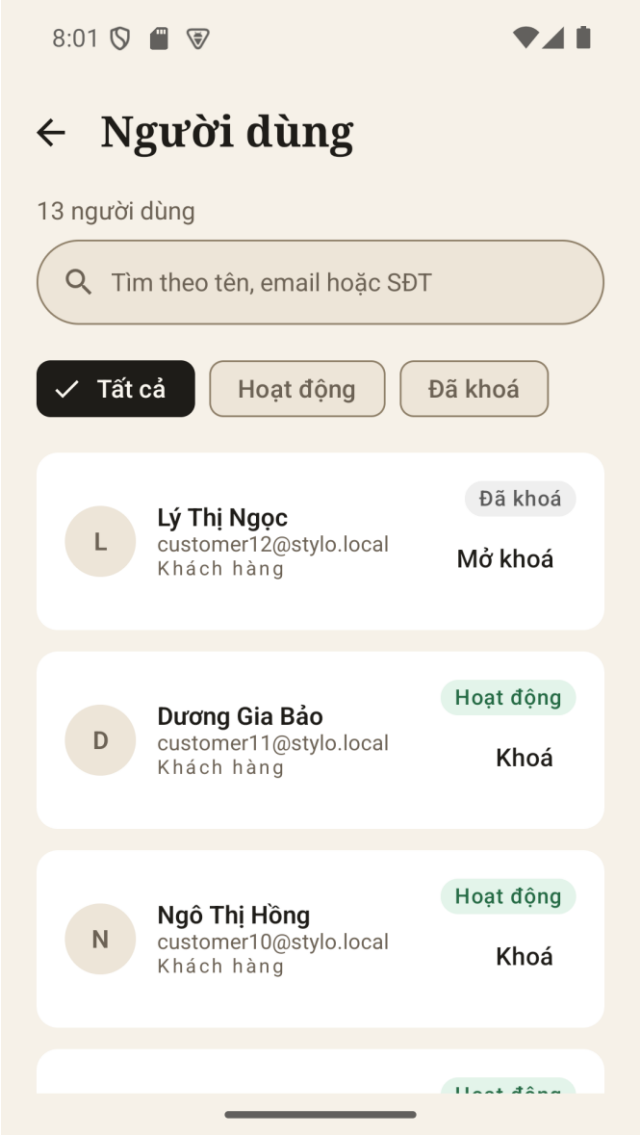
Hình 4.22. Báo cáo doanh thu theo khoảng ngày



Hình 4.23. Báo cáo tồn thấp theo ngưỡng

4.5.6. Người dùng và danh mục

Danh sách người dùng có tìm kiếm theo tên hoặc email, nhãn vai trò và trạng thái, cùng hành động khoá hoặc mở khoá kèm hộp thoại xác nhận. Danh mục và thương hiệu dùng chung một kiểu màn hình: danh sách kèm trạng thái và biểu mẫu thêm mới.



Hình 4.24. Quản lý người dùng với trạng thái và hành động khoá

4.6. Khả năng tiếp cận

Ba yêu cầu được kiểm tra trên từng màn hình thay vì chỉ ghi trong tài liệu. **Tương phản**: mọi cặp chữ/nền đạt tối thiểu 4.5:1 cho chữ thường và 3:1 cho chữ lớn; viền control dùng `outline` đạt 3.63:1 trên nền `surface`, 3.42:1 trên `background` và 3.08:1 trên `surfaceVariant`, đủ cho yêu cầu 3:1 của WCAG 1.4.11; màu `outlineVariant` không được dùng làm viền control vì chỉ đạt 1.28:1. **Không chỉ dựa vào màu**: chip đang chọn có thêm icon kiểm, trạng thái đơn có nhãn chữ, tồn thấp có chữ "(tồn thấp)" bên cạnh màu cảnh báo. **Vùng chạm**: mọi nút và chip cao tối thiểu 48dp; danh sách giữ nguyên kích thước khi phóng to chữ hệ thống nhờ dùng `sp` và bố cục co giãn.

4.7. Đợt đánh giá và những gì đã thay đổi

Sau khi hoàn thiện các màn hình, một đợt đánh giá có hệ thống được chạy trên hai mươi mốt màn hình, đối chiếu với tài liệu thiết kế và Material 3. Kết quả: ba lỗi mức P0, hai mươi bốn mức P1 và khoảng ba mươi mức P2. Nguyên nhân gốc của phần lớn lỗi không phải từng màn hình riêng lẻ mà là việc thiếu một lớp thành phần dùng chung: mỗi màn tự dựng chip, thẻ và nhãn trạng thái theo cách riêng, nên sửa một lỗi ở một màn không lan sang các màn còn lại.

Sáu quyết định được chốt và thi công trong một lần sửa (ADR-0016): thẻ dùng `surfaceContainerLowest` thay vì `surface`; bỏ đổ bóng và viền khỏi thẻ; đưa mọi điều khiển lặp lại vào lớp thành phần chung; thay menu tràn của quản trị bằng ngăn kéo; bỏ chỉ số "Đã bán N" vì API không có endpoint tổng hợp; và đổi nhãn nút mua để nêu kích cỡ. Sau khi sửa, giao diện chỉ còn một cách hiển thị cho mỗi loại điều khiển, và các màn hình đều dùng chung bốn trạng thái dữ liệu ở mục 4.2.5.

5. CÀI ĐẶT VÀ KIỂM THỬ

5.1. Kiến trúc tổng thể

Kho mã nguồn gồm hai dự án build độc lập và một thư mục tài liệu dùng chung:

```
stylo-clothing-store/
├── android/        ứng dụng Android, một module Gradle
├── server/         API Node.js, Prisma, MySQL
├── docs/           đặc tả, ADR, tài liệu thiết kế
└── GLOSSARY.md     thuật ngữ dùng xuyên suốt
```

Hai dự án không chia sẻ mã. Chúng chia sẻ một thứ duy nhất: hợp đồng API. Đây là lựa chọn có chủ đích cho một đồ án một người, vì nó biến mọi thay đổi giao diện thành thay đổi cục bộ và buộc mọi thay đổi dữ liệu phải đi qua một chỗ có kiểm thử. Đường đi của một yêu cầu điển hình: màn hình Compose gọi ViewModel, ViewModel gọi repository, repository gọi API qua Retrofit; phía server, route kiểm tra đầu vào bằng Zod, gọi service, service mở giao dịch và gọi repository, repository nói chuyện với Prisma.

5.2. Phía server

5.2.1. Công nghệ và phiên bản

Bảng 5.1. Thành phần phía server

Thành phần	Phiên bản	Vai trò
Node.js	24	Môi trường chạy, yêu cầu tối thiểu 22.12
Express	5.3.0	Định tuyến HTTP
Prisma	7.10.1	ORM và migration, client sinh từ lược đồ
Prisma adapter MariaDB	7.10.1	Driver thật cho MySQL 8.4
MySQL	8.4	Cơ sở dữ liệu, chạy trong container
Zod	4.6.7	Kiểm tra và kiểu hoá dữ liệu vào
argon2	0.45.2	Băm mật khẩu
Vitest	5.1	Kiểm thử đơn vị và tích hợp
Supertest	7.3	Gọi HTTP trong kiểm thử, không mở cổng

5.2.2. Luật module

Mỗi nhóm chức năng nằm trong một thư mục với bốn loại tệp có trách nhiệm tách bạch: `*.routes.ts` đọc HTTP và gọi đúng một hàm service; `*.service.ts` giữ quy tắc nghiệp vụ, mở giao dịch và quyết định mã lỗi; `*.repository.ts` chỉ chứa truy vấn Prisma; `*.schema.ts` chứa Zod schema của đầu vào. Ba luật đi kèm, và cả ba đều được kiểm bằng cách đọc mã khi review: service không import kiểu của Express, repository không bao giờ ném lỗi nghiệp vụ, và kiểu đầu vào chỉ được suy ra từ Zod (`z.infer`). Nhờ vậy không có hai nguồn sự thật cho hình dạng dữ liệu vào.

Module xuất hàm thuần, không dùng lớp và không có container DI ở server. Điều này giữ đường gọi ngắn: route nhìn thấy service, service nhìn thấy repository, repository nhìn thấy một Prisma client duy nhất.

5.2.3. Mô hình lỗi

Mọi lỗi trả về cùng một hình dạng: `{ error: { code, message, details? } }`. `code` là hằng ổn định (`VALIDATION_FAILED`, `OUT_OF_STOCK`, `INVALID_ORDER_STATE`, `ACCOUNT_DISABLED`, …), `message` là câu tiếng Anh dành cho log và lập trình viên, `details` có hình dạng cố định theo từng mã. Ứng dụng Android dịch `code` sang tiếng Việt và không bao giờ đọc `message`; nhờ vậy đổi câu chữ ở server không làm hỏng giao diện, còn thêm một mã lỗi mới thì buộc phải thêm một chuỗi tiếng Việt.

Một điểm kỹ thuật đáng ghi lại: khi dùng Prisma với driver MariaDB, lỗi khoá ngoại và lỗi trùng khoá đến thẳng từ driver dưới dạng mã `ER_ROW_IS_REFERENCED_2`, `ER_NO_REFERENCED_ROW_2`, `ER_DUP_ENTRY`, không phải lúc nào cũng thành `P2002`/`P2003` của Prisma. Vì vậy service kiểm tra cả hai họ mã trước khi quyết định trả `IN_USE` hay `CONFLICT`; nếu chỉ kiểm tra mã Prisma, người dùng sẽ nhận lỗi 500 ở đúng những tình huống đã được thiết kế để trả lỗi 409.

5.2.4. Cấu hình và chạy

Cấu hình đọc từ biến môi trường và được kiểm bằng Zod ngay khi khởi động, nên thiếu biến bắt buộc là lỗi rõ ràng lúc chạy chứ không phải lỗi khó hiểu về sau. Cổng mặc định của API là 8787; lý do chọn cổng này là cổng 3000 trên máy phát triển đã bị một ứng dụng khác chiếm. Android bản debug trỏ tới `http://127.0.0.1:8787`

và đi qua cầu `adb reverse`, nên máy ảo gọi được API trên máy thật mà không cần cấu hình mạng.

5.3. Hợp đồng API

5.3.1. Quy ước

Bảng 5.2. Quy ước của hợp đồng API

Quy tắc	**Giá trị**
Đường dẫn gốc	`/api/v1`
Tên trường	`camelCase` trong JSON; `snake_case` chỉ trong cơ sở dữ liệu
Tiền	Số nguyên VND, không chuỗi, không phần thập phân
Thời gian	ISO-8601 UTC kèm mili giây
Ngày lịch	`YYYY-MM-DD`, hiểu theo `Asia/Ho_Chi_Minh`
Giá trị thiếu	Trường vẫn có mặt với `null`, không bị bỏ
Trường lạ trong body	Bị từ chối bằng `VALIDATION_FAILED`
Thành công	Trả chính tài nguyên, hoặc `{ items, page, limit, total }` cho danh sách
Phân trang	`page` từ 1, `limit` 1–100, mặc định 20; `total` là tổng số dòng khớp
Sắp xếp	Mỗi endpoint có tập giá trị `sort` riêng; luôn kết thúc bằng `id` để ổn định
Xác thực	`Authorization: Bearer <accessToken>`

5.3.2. Nhóm endpoint

Toàn bộ danh sách endpoint nằm ở phụ lục A; bảng dưới chỉ nêu nhóm và số lượng để thấy mức phủ.

Bảng 5.3. Các nhóm endpoint của API

Nhóm	**Endpoint**	**Ghi chú**
Xác thực và hồ sơ	6	Đăng ký, đăng nhập, làm mới, đăng xuất, xem và sửa hồ sơ
Địa chỉ	4	Danh sách, thêm, sửa, xoá; đặt mặc định qua `PATCH`
Danh mục công khai	3	Danh sách sản phẩm, chi tiết, bộ lọc
Giỏ hàng	5	Xem, thêm, sửa, xoá dòng, xoá tất cả

Nhóm	Endpoint	Ghi chú
Đơn hàng	4	Đặt, danh sách, chi tiết, huỷ
Thanh toán	4	Tạo giao dịch, tra giao dịch, hai endpoint cho cổng
Quản trị	30	Danh mục, thương hiệu, sản phẩm, biến thể, ảnh, đơn, kho, người dùng, báo cáo

5.3.3. Hai cơ chế bảo vệ

Idempotency khi đặt hàng. Mỗi lần bấm đặt hàng, ứng dụng sinh một `clientOrderId` và gửi kèm. Server lưu cặp (`user_id`, `client_order_id`) là duy nhất; nếu nhận lại cùng cặp, server trả về đơn đã tạo thay vì tạo đơn thứ hai. Cơ chế này giải quyết tình huống thực tế nhất của mạng di động: yêu cầu đến được server, phản hồi bị mất, người dùng bấm lại. Không có nó, mỗi lần mạng chập chờn là một đơn hàng và một lần trừ tồn.

Xác nhận thanh toán muộn. Với thanh toán online, trạng thái đơn không được đặt từ tham số trên đường dẫn trình duyệt quay về, vì tham số đó do phía client mang tới. Server xác minh chữ ký và số tiền với dữ liệu giao dịch đã lưu rồi mới đổi trạng thái (ADR-0007). Điều này quan trọng ở môi trường sandbox nơi IPN không gọi được vào máy nội bộ: vẫn có một đường xác nhận đúng, thay vì tin vào URL.

5.4. Phía Android

5.4.1. Kiến trúc và thư viện

Một module `:app`, `minSdk 26`, `compileSdk` và `targetSdk` 36, Java 17. Mô hình MVVM cộng repository; mỗi feature là một gói gồm `ui`, `domain` (nếu có) và `data`. Trạng thái màn hình là một `data class` bất biến, hiệu ứng một lần (điều hướng, snackbar) đi qua kênh riêng, và ViewModel chỉ phát ra dữ liệu — không chuỗi tiếng Việt nào nằm trong ViewModel.

Bảng 5.4. Thành phần phía Android

Thành phần	Phiên bản	Vai trò
Kotlin	2.3.21	Ngôn ngữ
Android Gradle Plugin	8.13.2	Build

Thành phần	Phiên bản	Vai trò
Compose BOM	2026.03.00	Bộ thư viện UI
Hilt	2.58	Tiêm phụ thuộc
Navigation Compose	2.9.6	Điều hướng
Retrofit	3.0.0 + OkHttp 4.12	Tầng mạng
kotlinx.serialization	1.12.0	JSON
Coil	3.0.4	Tải ảnh
DataStore	1.2.0	Lưu token
Kaspresso	1.6.1	Kiểm thử giao diện

5.4.2. Tầng mạng và phiên đăng nhập

Mọi lời gọi API trả về một kiểu kết quả có ba nhánh: thành công, lỗi nghiệp vụ (đã dịch sang mã và thông điệp), và lỗi hạ tầng (mạng, timeout). Nhờ vậy ViewModel không phải bắt ngoại lệ, và lỗi mạng không bị hiển thị như lỗi nghiệp vụ.

Access token sống 15 phút; refresh token 30 ngày và được xoay mỗi lần dùng. Một interceptor gắn token vào mỗi yêu cầu; một authenticator chặn lời gọi khi gặp 401, làm mới token đúng một lần cho nhiều yêu cầu song song, rồi chạy lại yêu cầu ban đầu. Nếu làm mới thất bại, phiên bị xoá và người dùng về màn hình đăng nhập. Token được lưu trong DataStore, không lưu trong bộ nhớ tĩnh.

5.4.3. Tầng giao diện

Một theme duy nhất, một lớp thành phần dùng chung (chương 4). Điều hướng mount theo vai trò: đồ thị quản trị chỉ được thêm vào khi vai trò là `ADMIN`. Các danh sách dùng `LazyColumn` với khoá ổn định, và trạng thái cuộn của tab được lưu khi đổi tab.

Điểm đáng nêu vì nó là nguồn lỗi phổ biến: ứng dụng không giữ bản sao dữ liệu trong ViewModel lâu hơn màn hình. Sau khi đặt hàng, giỏ được đọc lại từ server thay vì xoá cục bộ; sau khi huỷ đơn, danh sách đơn được tải lại. Cách này đắt hơn một chút về số lời gọi nhưng loại bỏ hoàn toàn lớp lỗi "giao diện nói một đằng, server nói một nẻo".

5.5. Thanh toán

5.5.1. Ranh giới trừu tượng

Cổng thanh toán nằm sau một interface duy nhất (`PaymentGateway`) với hai hiện thực: `MockGateway` và `VNPayGateway`. Chọn cổng bằng biến môi trường `PAYMENT_PROVIDER`. Service đặt hàng không biết cổng nào đang chạy; nó chỉ tạo dòng `payment` và hỏi cổng lấy URL thanh toán. Nhờ vậy môi trường phát triển và demo dùng cổng mock, còn sandbox dùng VNPay, mà không có nhánh `if` nào trong nghiệp vụ.

5.5.2. Cổng mock

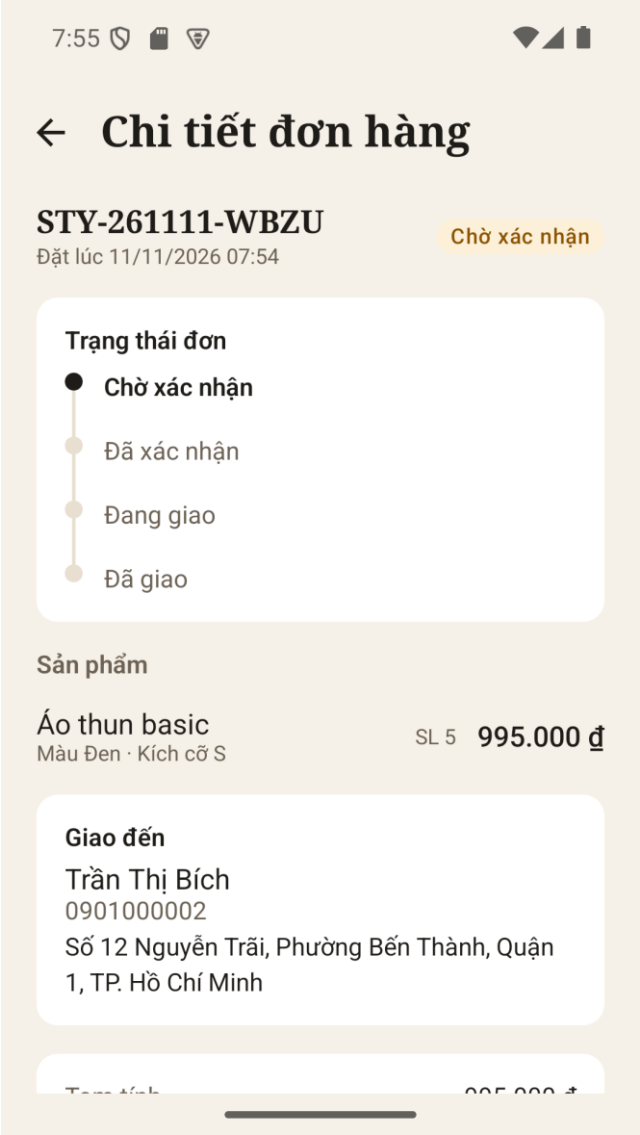
Cổng mock dựng một trang HTML do chính API phục vụ, hiển thị số tiền và hai lựa chọn "thanh toán thành công" hoặc "thanh toán thất bại". Mỗi lựa chọn gọi endpoint xác nhận của mock với đúng `txn_ref` của giao dịch. Trang này là công cụ demo: nó cho thấy toàn bộ đường đi của thanh toán — tạo giao dịch, mở trình duyệt, xác nhận, quay lại ứng dụng — mà không phụ thuộc dịch vụ ngoài.

5.5.3. Cổng VNPay

Adapter VNPay dựng URL thanh toán theo đặc tả sandbox: sắp xếp tham số theo tên, nối thành chuỗi truy vấn, ký bằng HMAC-SHA512 với khoá bí mật, và đính kèm chữ ký. URL có thêm `vnp_ExpireDate` bằng thời điểm tạo giao dịch cộng 15 phút, trùng với hạn của lần thử thanh toán, để cổng không giữ một giao dịch đã quá hạn ở phía server. Chiều ngược lại, khi cổng gọi về, adapter kiểm tra chữ ký trên toàn bộ tham số nhận được, đối chiếu số tiền với dòng `payment` đã lưu, rồi mới xác nhận.

Trạng thái kiểm chứng với sandbox thật: unit test và integration test của adapter và hai endpoint callback đều đạt; gọi thẳng trang thanh toán với terminal code sai trả mã 72, với khoá bí mật sai trả mã 70, còn request đúng chữ ký của terminal trong cấu hình vẫn nhận trang lỗi chung. Ba kết quả đó chỉ ra rằng adapter ký đúng và terminal có tồn tại, nhưng terminal chưa được duyệt hoặc Return URL đã đăng ký chưa khớp cấu hình trên cổng merchant — một việc phải xử lý ở portal, không sửa được bằng mã nguồn. Vì vậy hồ sơ giữ `PAYMENT_PROVIDER=MOCK` cho demo và ghi lại toàn bộ phép thử trong `docs/vnpay.md`.

Giới hạn đã biết: IPN của VNPay không gọi được vào một server chạy trên máy cá nhân không có địa chỉ công cộng, nên đường xác nhận trong demo là đường quay về trình duyệt kèm kiểm tra chữ ký. Đây không phải chỗ đánh đổi chất lượng: chữ ký vẫn được kiểm, số tiền vẫn được đối chiếu, và bản ghi giao dịch vẫn là nguồn sự thật. Chỉ khác là thông báo đến sớm hay muộn mà thôi.



Hình 5.1. Đơn hàng sau khi đặt: trạng thái, sản phẩm, địa chỉ, tạm tính

5.6. Năm báo cáo

Báo cáo là phần dễ sai lệch nhất giữa "con số trên màn hình" và "con số trong cơ sở dữ liệu", nên mỗi báo cáo được định nghĩa bằng cột ngày dùng để lọc và bằng công thức, không bằng cảm nhận.

Bảng 5.5. Định nghĩa năm báo cáo

Báo cáo	Cột ngày	Công thức	Ghi chú
Doanh thu	`order.paid_at`	Tổng `order.total` của đơn đã thanh toán trong kỳ	Gồm phí vận chuyển; lọc theo ngày Việt Nam
Sản phẩm bán chạy	`order.paid_at`	Tổng `quantity` theo sản phẩm, sắp giảm dần	Cùng tập đơn với báo cáo doanh thu
Đơn theo trạng thái	`order.created_at`	Đếm đơn theo từng trạng thái	Đủ năm trạng thái, kể cả trạng thái không có đơn
Tồn thấp	không có	Các biến thể có `stock` dưới ngưỡng	Là ảnh chụp hiện tại, không phải theo kỳ
Chi phí và lãi	`paid_at` cho giá vốn, `inventory_receipt.created_at` cho tiền nhập	`revenue − cogs`, kèm tổng tiền nhập	Hai dòng tiền khác nhau nên tách cột

Ba quy tắc áp dụng cho cả năm: khoảng ngày là bao gồm hai đầu và hiểu theo giờ Việt Nam (UTC+7, không có giờ mùa hè); kỳ trả về luôn được lấp đầy, tức mọi ngày trong khoảng đều có mặt kể cả ngày không có đơn, để biểu đồ không bị co lại thành những cột dồn cục; và việc lấp đầy làm ở tầng service chứ không ở SQL, để truy vấn giữ được chỉ mục.

Doanh thu của đơn COD ghi nhận ở ngày giao, không phải ngày đặt. Đây là hệ quả trực tiếp của định nghĩa "doanh thu là tiền đã nhận", và nó được kiểm thử bằng một đơn đặt trước nhưng giao sau, nằm ở hai kỳ khác nhau.

5.7. Chiến lược kiểm thử

5.7.1. Bốn tầng

Bảng 5.6. Các tầng kiểm thử

Tầng	Công cụ	Phạm vi	Mục tiêu thời gian
Đơn vị backend	Vitest, không cơ sở dữ liệu	Hàm thuần: tiền, chuyển trạng thái, ngày, chữ ký VNPay	dưới 5 giây
Tích hợp backend	Vitest + Supertest + MySQL thật	Đường HTTP, service, giao dịch, SQL, sổ cái	dưới 60 giây

Tầng	Công cụ	Phạm vi	Mục tiêu thời gian
JVM Android	JUnit4, coroutines-test, Turbine	Trạng thái ViewModel, mapper, giải mã DTO, câu lỗi	dưới 30 giây
Giao diện Android	Kaspresso 1.6.1 trên máy ảo	Đăng ký, đăng nhập, danh mục, hồ sơ, địa chỉ, giỏ, COD, thanh toán online, đơn hàng	vài phút

Nguyên tắc chọn tầng: kiểm thử bất biến ở nơi một lỗi gây mất tiền hoặc sai tồn, không kiểm thử từng route. Prisma không bao giờ bị mock ở tầng tích hợp — mọi kiểm thử tích hợp chạy trên MySQL thật với lược đồ thật, vì phần lớn lỗi của hệ thống này nằm ở giao dịch và ràng buộc, không nằm ở tầng HTTP.

5.7.2. Cách ly dữ liệu kiểm thử

Kiểm thử tích hợp dùng một cơ sở dữ liệu riêng tên `stylo_test`, và global setup dừng ngay nếu tên không khớp. Trước mỗi tệp kiểm thử, toàn bộ bảng bị truncate với `FOREIGN_KEY_CHECKS = 0`; mỗi ca kiểm thử tạo dữ liệu của riêng nó qua factory với email, SKU và mã đơn duy nhất. Không có phụ thuộc thứ tự, không có ID cố định, và không có `sleep`: thời gian được mô phỏng bằng cách sửa trực tiếp các cột thời gian khi cần kiểm tra hết hạn hoặc cửa sổ báo cáo.

5.7.3. Kiểm thử giao diện

Năm hành trình Kaspresso, mỗi tệp một hành trình: khách vãng lai duyệt và tìm kiếm; khách hàng đăng ký, đăng nhập, sửa hồ sơ, thêm địa chỉ; giỏ hàng và đặt COD; thanh toán online qua cổng mock; danh sách và chi tiết đơn kèm huỷ đơn. Chúng chạy trên máy ảo `stylo_api35` với server thật và dữ liệu seed, đúng như buổi demo.

Một chi tiết vận hành đáng ghi: trên máy phát triển này, cổng 5037 mặc định của adb nằm trong dải cổng bị Windows giữ, nên cả máy ảo, adb và Gradle đều được chạy với biến `ANDROID_ADB_SERVER_PORT=5359`. Ngoài ra công cụ điều khiển thiết bị dùng chung kết nối UiAutomation với Kaspresso, nên daemon của nó phải được dừng trước khi chạy kiểm thử giao diện; cả hai điều kiện được gói trong một script chạy duy nhất (`android/e2e.cmd`).

Một điều kiện nữa chỉ lộ ra khi chạy thật: bộ kiểm thử giả định app bắt đầu ở trạng thái khách, nên nếu lần chạy trước kết thúc trong phiên quản trị thì bước đầu tiên

treo cho tới hết thời gian chờ. Cách xử lý là xoá dữ liệu app trước khi chạy (`adb shell pm clear com.stylo.shop`), không phải nới thời gian chờ — nới ra chỉ giấu đi việc app đang ở sai màn hình.

5.8. Kết quả lần chạy cuối

Số liệu dưới đây là của lần chạy tại thời điểm cập nhật hồ sơ (2026-11-11), trên máy phát triển nêu ở phụ lục B. Không có số liệu nào được làm tròn hay suy đoán.

Bảng 5.7. Kết quả kiểm thử của lần chạy cuối

Bộ kiểm thử	Số tệp	Số ca	Kết quả	Thời gian
Đơn vị backend	7	57	57 đạt	1,33 giây
Tích hợp backend	13	198	198 đạt	527 giây
JVM Android	33 lớp	195	195 đạt	19 giây
Giao diện Android	5	5	5 đạt	137 giây

Một lỗi thật đã được phát hiện và sửa trong chính lần chạy này, và nó đáng được ghi lại vì nó thuộc loại lỗi chỉ lộ ra khi dữ liệu lớn lên: ca kiểm thử chạy seed đầy đủ rồi truy vấn qua endpoint công khai bị quá thời gian chờ 30 giây của bộ kiểm thử. Nguyên nhân không phải seed chậm bất thường mà là seed đã lớn — 185 sản phẩm, 1.330 biến thể, 140 đơn, 2.723 dòng phiếu nhập — trong khi hạn mức vẫn là con số đặt ra lúc danh mục còn nhỏ. Cách sửa là cho riêng ca kiểm thử đó một hạn mức 180 giây kèm ghi chú giải thích, thay vì nâng hạn mức toàn bộ (điều sẽ che mất các ca chậm bất thường khác). Sau khi sửa, cả 198 ca đạt. Toàn bộ bộ tích hợp chạy 527 giây, lâu hơn lần đóng băng trước vì mỗi ca dùng seed phải tạo nhiều biến thể hơn.

5.8.1. Dữ liệu seed

Dữ liệu demo được sinh bằng một script có tính tất định: chạy lại cho ra đúng tập dữ liệu đó, cùng thứ tự, cùng SKU. Nguồn là một bộ dữ liệu quần áo công khai, được lọc bỏ giày dép và chỉnh lại mô tả tiếng Việt.

Bảng 5.8. Thành phần dữ liệu seed

Thực thể	Số lượng	Ghi chú

Thực thể	Số lượng	Ghi chú
Danh mục	4	Ba danh mục đang bán (Áo, Quần, Áo khoác) và một danh mục ẩn (Phụ kiện)
Thương hiệu	4	Stylo, Basic Lab, Urban Viet, Local Brand
Sản phẩm	185	95 áo, 60 quần, 30 áo khoác; 162 đang bán, 23 đã ẩn; 154 sản phẩm còn hàng
Biến thể	1.330	Mỗi biến thể một SKU, kích cỡ và màu riêng; mỗi sản phẩm 2–3 màu (104 sản phẩm hai màu, 81 ba màu), 3–12 biến thể
Ảnh sản phẩm	185	Mỗi sản phẩm một ảnh, tệp cục bộ
Người dùng	13	Một quản trị và mười hai khách, trong đó một khách bị khoá
Đơn hàng	140	Trải trên cả năm trạng thái, có đơn COD và đơn online; 292 dòng đơn và 73 lần thử thanh toán
Phiếu nhập kho	3	2.723 dòng, đủ để tính giá vốn bình quân
Dòng sổ cái tồn	3.045	Khớp tuyệt đối với tồn hiện tại của mọi biến thể

Tính đúng của sổ cái được kiểm bằng một truy vấn đối chiếu: tổng `delta` theo từng biến thể phải bằng `stock`. Trên dữ liệu seed, số dòng lệch là 0.

5.8.2. Kiểm chứng hồ sơ nộp

Ba kiểm chứng dưới đây được chạy trên chính các tệp nộp kèm, không phải trên mã nguồn:

- **Tệp dump** được nạp vào một lược đồ rỗng, sau đó API được khởi động trỏ vào lược đồ đó: `/health`

trả `ok`, danh sách sản phẩm trả 154 sản phẩm đang bán với trang đầu 20 dòng, bộ lọc trả đúng ba danh mục đang bán, 4 thương hiệu, 18 màu và chín kích cỡ. Toàn bộ đếm dòng của 17 bảng khớp với cơ sở dữ liệu gốc.

- **Sơ đồ ERD** được sinh từ lược đồ Prisma, không vẽ tay, nên nó khớp với cơ sở dữ liệu đang chạy.

• **Báo cáo PDF** được xuất từ Word với font nhúng, mục lục cập nhật, và số trang thân bài nằm trong
khoảng yêu cầu.

5.9. Hồ sơ nộp

Bảng 5.9. Các tệp trong hồ sơ nộp

Tệp	Nội dung	Cách sinh
`BaoCao_Stylo.pdf` và `.docx`	Báo cáo này	Sinh từ nguồn Markdown bằng script, xuất PDF qua Word
`Slide_BaoVe.pptx` và `.pdf`	Slide bảo vệ 15 phút	Sinh từ script, xuất PDF qua PowerPoint
`ERD_Stylo_A3.pdf`	Sơ đồ ERD một trang A3	Sinh từ lược đồ Prisma, in A3 ngang
`stylo-db.sql`	Dump cơ sở dữ liệu sau seed	`mysqldump --databases stylo`
`Stylo_Source.zip`	Mã nguồn, không gồm thư viện và tệp môi trường	Kết xuất từ git
`Demo_Stylo.mp4`	Video demo 5–10 phút trên máy ảo	Quay màn hình thiết bị

6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Đối chiếu với yêu cầu đề bài

Mười một chức năng đề bài yêu cầu đều đã được cài đặt, có giao diện và có đường kiểm chứng. Bảng dưới đối chiếu từng mục với nơi thực hiện và cách kiểm chứng, để người đọc không phải suy luận từ mô tả.

Bảng 6.1. Đối chiếu mười một chức năng của đề bài

Chức năng đề bài	Nơi thực hiện	Kiểm chứng
Đăng ký, đăng nhập	Màn Đăng nhập/Đăng ký; `POST /auth/register`, `POST /auth/login`	Tích hợp xác thực, JVM cho ViewModel, hành trình Kaspresso
Quản lý thông tin cá nhân	Màn Tôi, Địa chỉ giao hàng	Tích hợp hồ sơ và địa chỉ, JVM, Kaspresso
Quản lý giỏ hàng	Màn Giỏ hàng; năm endpoint giỏ	Tích hợp giỏ, Kaspresso, kiểm tra chặn quá tồn
Quản lý sản phẩm	Màn Sản phẩm của quản trị; nhóm endpoint sản phẩm, biến thể, ảnh	Tích hợp quản trị sản phẩm và tải ảnh
Quản lý đơn hàng	Màn Đơn hàng hai phía; endpoint đơn và chuyển trạng thái	Tích hợp vòng đời đơn, Kaspresso danh sách và huỷ đơn
Quản lý người dùng	Màn Người dùng; khoá và mở khoá	Tích hợp quản trị người dùng
Tìm kiếm sản phẩm	Màn Tìm kiếm, sheet bộ lọc; `GET /products`, `GET /catalog/filters`	Tích hợp danh mục, Kaspresso tìm kiếm không dấu
Chi tiết sản phẩm	Màn Chi tiết với chọn biến thể	Tích hợp chi tiết, JVM cho trạng thái chọn biến thể
Thanh toán	Màn Thanh toán; COD, mock, VNPay	Tích hợp thanh toán và báo cáo doanh thu, Kaspresso thanh toán online
Báo cáo và thống kê	Màn Báo cáo với năm báo cáo	Tích hợp báo cáo theo cửa sổ ngày
Phiếu giá nhập kho	Màn Kho; phiếu nhập nhiều dòng và lịch sử giá nhập	Tích hợp kho và sổ cái tồn

6.2. Hạn chế đã biết

Không có hệ thống nào hoàn thiện; những hạn chế dưới đây là hạn chế có ý thức, được nêu để người đọc đánh giá đúng mức độ hoàn thiện chứ không phải để che đi phần còn thiếu.

Chưa có đổi trả và hoàn tiền. Đơn đã thanh toán không huỷ được, và không có luồng trả hàng. Trong thực tế đây là nghiệp vụ phổ biến nhất bị hỏi đầu tiên, và nó kéo theo yêu cầu về đối soát với cổng thanh toán. Hệ thống hiện tại cố tình chặn ở điều kiện `paid_at IS NULL` để không tạo ra một trạng thái trả hàng nửa vời.

Chưa có giỏ hàng cho khách vãng lai. Người chưa đăng nhập phải đăng nhập trước khi thêm vào giỏ. Đổi lại, hệ thống không phải giải bài toán gộp giỏ giữa nhiều thiết bị và không phải quản lý giỏ rác.

Chưa có quên mật khẩu và xác thực email. Cả hai đều cần dịch vụ gửi thư, một thành phần ngoài phạm vi đồ án. Hệ quả: tài khoản tạo ra là dùng được ngay, nhưng không có đường tự phục hồi khi quên.

IPN của VNPay không chạy trong môi trường nội bộ. Đường xác nhận trong demo là đường quay về kèm kiểm tra chữ ký. Với một hệ thống chạy thật, IPN là đường chính và đường quay về chỉ là đường dự phòng.

Trang thanh toán sandbox chưa mở được. Adapter ký đúng — gọi với terminal code sai trả mã 72, khoá bí mật sai trả mã 70 — nhưng request đúng cấu hình vẫn nhận trang lỗi chung, tức terminal chưa được duyệt hoặc Return URL đã đăng ký chưa khớp. Đây là việc ở portal merchant, không phải lỗi mã nguồn; hồ sơ vì vậy giữ cổng mock cho demo và ghi lại phép thử trong `docs/vnpay.md`.

Tìm kiếm dùng `LIKE`. Với 185 sản phẩm, cách này nhanh và đơn giản. Với vài chục nghìn sản phẩm, nó sẽ là điểm nghẽn đầu tiên; khi đó cần chỉ mục toàn văn hoặc một cột tên đã chuẩn hoá.

Không có CI. Toàn bộ kiểm thử chạy trên máy phát triển bằng tay. Một đường ống CI sẽ là việc đầu tiên nên làm nếu dự án có thêm người thứ hai.

Chỉ kiểm chứng trên một máy ảo. Giao diện được kiểm trên `stylo_api35` (Android 15, 360×640dp). Các kích thước màn hình khác, đặc biệt máy tính bảng và màn hình gập, chưa được kiểm.

6.3. Hướng phát triển

Bốn hướng dưới đây được xếp theo tỉ lệ giữa giá trị mang lại và công sức bỏ ra, không theo mức độ hấp dẫn kỹ thuật.

Thêm luồng đổi trả có đối soát. Đây là phần bù tự nhiên cho quy tắc "đơn đã thanh toán không huỷ được". Cần một trạng thái trả hàng, một bản ghi hoàn tiền và một quy tắc hoàn tồn chỉ áp dụng khi hàng về kho, vì hàng hoàn có thể hỏng.

Đường ống CI chạy bốn bộ kiểm thử. Ba bộ chạy được trên máy không cần thiết bị; bộ Kaspresso cần một máy ảo trong CI. Chỉ cần một workflow nối các lệnh đã có trong `docs/testing.md`.

Tìm kiếm có xếp hạng. Thay `LIKE` bằng chỉ mục toàn văn, thêm trọng số cho tên sản phẩm và thương hiệu, và ghi nhận từ khoá không có kết quả để chỉnh danh mục. Bước này chỉ nên làm khi dữ liệu đủ lớn để đo được cải thiện.

Thông báo đẩy cho trạng thái đơn. Người mua hiện phải mở ứng dụng để biết đơn đã giao. Một thông báo đẩy khi đơn chuyển sang `SHIPPING` hoặc `DELIVERED` là thay đổi nhỏ ở server và vừa phải ở phía Android, và nó cải thiện trải nghiệm rõ hơn nhiều so với việc thêm màn hình mới.

6.4. Bài học

Ba bài học dưới đây đến từ những chỗ đồ án thực sự vấp, không phải từ danh sách nguyên tắc chung.

Đặc tả trước giúp sửa sai rẻ hơn, nhưng chỉ khi đặc tả được kiểm. Quyết định "tiền là số nguyên VND" được ghi thành ADR từ đầu, nên không có đoạn mã nào phải sửa khi làm báo cáo. Ngược lại, quy ước "mọi thay đổi tồn đều có dòng sổ cái" chỉ có giá trị vì nó được kiểm bằng một truy vấn đối chiếu chạy trên dữ liệu seed; nếu không có truy vấn đó, nó sẽ chỉ là một câu trong tài liệu.

Kiểm thử đắt nhất là kiểm thử ở ranh giới tiền và tồn. Phần lớn lỗi được phát hiện trong quá trình làm nằm ở ba chỗ: trừ tồn hai lần khi hai yêu cầu đến cùng lúc, ghi doanh thu sai ngày với đơn COD, và đơn trùng khi mạng chập chờn. Cả ba đều không phải lỗi giao diện, và cả ba đều được chặn ở tầng giao dịch hoặc ở một khoá duy nhất trong cơ sở dữ liệu.

Lớp thành phần dùng chung rẻ hơn sửa từng màn. Đợt đánh giá giao diện cho thấy cùng một lỗi tương phản màu lặp lại ở nhiều màn, và mỗi lần sửa lại sinh ra một biến thể mới. Sau khi gom về một lớp thành phần, việc sửa một quy tắc trở thành sửa một tệp. Bài học này áp dụng cho mọi dự án có nhiều màn hình hơn số người làm giao diện.

TÀI LIỆU THAM KHẢO

Các tài liệu dưới đây là nguồn được đọc và dùng trực tiếp trong quá trình làm đồ án. Tài liệu nội bộ nằm trong chính kho mã nguồn, ở thư mục `docs/`.

Tài liệu kỹ thuật

1. Prisma, *Prisma ORM documentation* — lược đồ, migration, giao dịch tương tác và driver adapter. 2. Express, *Express 5.x documentation* — định tuyến, middleware và xử lý lỗi bất đồng bộ. 3. Android Developers, *Compose documentation* — trạng thái, hiệu ứng một lần, `LazyColumn`, điều hướng. 4. Material Design 3, *Design tokens and components* — vai trò màu, thang chữ, thang hình dạng. 5. W3C, *Web Content Accessibility Guidelines 2.2* — tiêu chí 1.4.3 về tương phản và 1.4.11 về tương phản của thành phần giao diện. 6. Square, *OkHttp documentation* — interceptor và authenticator cho việc làm mới token. 7. VNPay, *Đặc tả tích hợp cổng thanh toán sandbox* — tham số, thứ tự sắp xếp và thuật toán ký.

Tài liệu nội bộ

8. `docs/PROJECT.md` — đề bài và môi trường phát triển. 9. `docs/erd.md` — lược đồ, quy ước dữ liệu, vòng đời đơn và bốn giao dịch then chốt. 10. `docs/api.md` — hợp đồng API, mô hình lỗi và danh mục mã lỗi. 11. `docs/ui.md` — hệ thống thiết kế, token và đặc tả từng màn hình. 12. `docs/architecture.md` — cấu trúc kho mã, luật module và cấu hình chạy. 13. `docs/testing.md` — chiến lược kiểm thử, ma trận bốn tầng và dữ liệu demo. 14. `docs/reports.md` — định nghĩa năm báo cáo, cột ngày và công thức. 15. `docs/adr/0001`–`0017` — mười bảy quyết định kiến trúc kèm bối cảnh và hệ quả. 16. `GLOSSARY.md` — bảng thuật ngữ dùng thống nhất trong mã, API và giao diện.

Phụ lục A. Danh sách endpoint theo module

Đường dẫn gốc là `/api/v1`. Cột "Quyền" ghi `Công khai` cho endpoint không cần token, `Đăng nhập` cho endpoint cần access token, và `Quản trị` cho endpoint cần vai trò `ADMIN`.

Bảng A.1. Endpoint xác thực và hồ sơ

Phương thức và đường dẫn	Quyền	Mục đích
`POST /auth/register`	Công khai	Tạo tài khoản khách, trả cặp token
`POST /auth/login`	Công khai	Đăng nhập, trả cặp token
`POST /auth/refresh`	Công khai	Đổi refresh token lấy cặp token mới
`POST /auth/logout`	Đăng nhập	Thu hồi refresh token hiện tại
`GET /me`	Đăng nhập	Hồ sơ của chính mình
`PATCH /me`	Đăng nhập	Sửa họ tên, số điện thoại

Bảng A.2. Endpoint địa chỉ giao hàng

Phương thức và đường dẫn	Quyền	Mục đích
`GET /me/addresses`	Đăng nhập	Danh sách địa chỉ, mặc định trước
`POST /me/addresses`	Đăng nhập	Thêm địa chỉ
`PATCH /me/addresses/:id`	Đăng nhập	Sửa địa chỉ hoặc đặt mặc định
`DELETE /me/addresses/:id`	Đăng nhập	Xoá địa chỉ

Bảng A.3. Endpoint danh mục công khai

Phương thức và đường dẫn	Quyền	Mục đích
`GET /products`	Công khai	Danh sách sản phẩm đang bán; `q`, `categoryId`, `brandId`, `color`, `size`, `minPrice`, `maxPrice`, `sort`, `page`, `limit`
`GET /products/:id`	Công khai	Chi tiết sản phẩm kèm biến thể và ảnh
`GET /catalog/filters`	Công khai	Danh mục, thương hiệu, màu, kích cỡ và khoảng giá đang có

Bảng A.4. Endpoint giỏ hàng

Phương thức và đường dẫn	Quyền	Mục đích
`GET /cart`	Đăng nhập	Giỏ hiện tại kèm tổng tiền
`POST /cart/items`	Đăng nhập	Thêm biến thể với số lượng
`PATCH /cart/items/:itemId`	Đăng nhập	Đổi số lượng
`DELETE /cart/items/:itemId`	Đăng nhập	Xoá một dòng
`DELETE /cart`	Đăng nhập	Xoá toàn bộ giỏ

Bảng A.5. Endpoint đơn hàng

Phương thức và đường dẫn	Quyền	Mục đích
`POST /orders`	Đăng nhập	Đặt hàng; body có `clientOrderId`, địa chỉ, phương thức thanh toán
`GET /orders`	Đăng nhập	Danh sách đơn của mình, lọc theo trạng thái
`GET /orders/:id`	Đăng nhập	Chi tiết một đơn của mình
`POST /orders/:id/cancel`	Đăng nhập	Huỷ đơn khi còn huỷ được

Bảng A.6. Endpoint thanh toán

Phương thức và đường dẫn	Quyền	Mục đích
`POST /orders/:id/payments`	Đăng nhập	Tạo lần thử thanh toán, trả URL cổng
`GET /payments/:txnRef`	Đăng nhập	Trạng thái một giao dịch
`GET /payments/vnpay/return`	Công khai	Cổng quay về trình duyệt; kiểm chữ ký rồi xác nhận
`GET /payments/vnpay/ipn`	Công khai	Thông báo giữa hai hệ thống; trả `RspCode` theo đặc tả

Bảng A.7. Endpoint quản trị danh mục và thương hiệu

Phương thức và đường dẫn	Quyền	Mục đích
`GET /admin/categories`	Quản trị	Danh sách, có tìm và lọc trạng thái
`POST /admin/categories`	Quản trị	Tạo danh mục
`PATCH /admin/categories/:id`	Quản trị	Đổi tên hoặc trạng thái

Phương thức và đường dẫn	Quyền	Mục đích
DELETE /admin/categories/:id	Quản trị	Xoá; 409 nếu còn sản phẩm
GET /admin/brands	Quản trị	Danh sách thương hiệu
POST /admin/brands	Quản trị	Tạo thương hiệu
PATCH /admin/brands/:id	Quản trị	Đổi tên hoặc trạng thái
DELETE /admin/brands/:id	Quản trị	Xoá; 409 nếu còn sản phẩm

Bảng A.8. Endpoint quản trị sản phẩm, biến thể và ảnh

Phương thức và đường dẫn	Quyền	Mục đích
GET /admin/products	Quản trị	Danh sách đầy đủ, lọc trạng thái, danh mục, thương hiệu
GET /admin/products/:id	Quản trị	Chi tiết kèm biến thể và ảnh
POST /admin/products	Quản trị	Tạo sản phẩm
PATCH /admin/products/:id	Quản trị	Sửa sản phẩm, kể cả ẩn hoặc hiện
DELETE /admin/products/:id	Quản trị	Xoá cứng; 409 nếu đã có chứng từ tham chiếu
POST /admin/products/:id/variants	Quản trị	Thêm biến thể
PATCH /admin/variants/:id	Quản trị	Sửa SKU, kích cỡ, màu, giá
DELETE /admin/variants/:id	Quản trị	Xoá biến thể chưa bị tham chiếu
GET /admin/variants	Quản trị	Tìm biến thể theo SKU hoặc tên, phục vụ phiếu nhập
POST /admin/products/:id/images	Quản trị	Tải một ảnh, tối đa 5 ảnh mỗi sản phẩm
DELETE /admin/products/:id/images/:imageId	Quản trị	Xoá ảnh và tệp tương ứng

Bảng A.9. Endpoint quản trị người dùng và đơn hàng

Phương thức và đường dẫn	Quyền	Mục đích
GET /admin/users	Quản trị	Danh sách, tìm theo tên hoặc email, lọc vai trò, trạng thái
PATCH /admin/users/:id/status	Quản trị	Khoá hoặc mở khoá tài khoản
GET /admin/orders	Quản trị	Hàng đợi đơn, lọc trạng thái, từ khoá, khoảng ngày

Phương thức và đường dẫn	Quyền	Mục đích
GET /admin/orders/:id	Quản trị	Chi tiết đơn kèm giá vốn từng dòng
PATCH /admin/orders/:id/status	Quản trị	Chuyển trạng thái theo bảng hợp lệ

Bảng A.10. Endpoint quản trị kho

Phương thức và đường dẫn	Quyền	Mục đích
POST /admin/inventory-receipts	Quản trị	Tạo phiếu nhập nhiều dòng, cộng tồn trong một giao dịch
GET /admin/inventory-receipts	Quản trị	Danh sách phiếu, lọc theo khoảng ngày
GET /admin/inventory-receipts/:id	Quản trị	Chi tiết phiếu kèm các dòng

Bảng A.11. Endpoint quản trị báo cáo

Phương thức và đường dẫn	Quyền	Mục đích
GET /admin/reports/revenue	Quản trị	Doanh thu theo ngày hoặc tháng
GET /admin/reports/top-products	Quản trị	Sản phẩm bán chạy theo số lượng
GET /admin/reports/orders-by-status	Quản trị	Số đơn theo từng trạng thái
GET /admin/reports/low-stock	Quản trị	Biến thể dưới ngưỡng tồn
GET /admin/reports/inventory-cost	Quản trị	Tiền nhập, giá vốn hàng bán và lãi gộp

Bảng A.12. Endpoint hệ thống

Phương thức và đường dẫn	Quyền	Mục đích
GET /health	Công khai	Kiểm tra API sống; dùng trong script kiểm thử giao diện
GET /uploads/...	Công khai	Phục vụ tệp ảnh sản phẩm đã tải lên

Phụ lục B. Kịch bản demo và môi trường

B.1. Kịch bản demo

Buổi demo chạy trực tiếp trên máy ảo với server nội bộ, tổng thời lượng khoảng mười lăm phút. Thanh toán dùng cổng mock làm đường chính, vì IPN của VNPay không gọi được vào server chạy trên máy cá nhân; adapter VNPay được trình bày ở mức mã nguồn và cấu hình.

Bảng B.1. Kịch bản demo theo mốc thời gian

Mốc	Việc làm	Điều cần thấy
0:00	Khách vãng lai: trang chủ, tìm "ao thun" không dấu, lọc theo thương hiệu, mở chi tiết	Tìm không dấu vẫn khớp; bộ lọc kết hợp được; chi tiết nêu tồn theo biến thể
3:00	Đăng nhập, sửa hồ sơ, thêm địa chỉ và đặt làm mặc định	Địa chỉ đầu tiên tự thành mặc định; danh sách cập nhật ngay
5:00	Thêm vào giỏ, tăng số lượng trong giỏ, đặt hàng COD	Bộ tăng giảm cập nhật tạm tính và tồn; đơn xuất hiện trong danh sách
8:00	Quản trị: xem tổng quan, xác nhận một đơn đang chờ	Hộp thoại nêu rõ cặp trạng thái; danh sách đổi trạng thái sau khi xác nhận
10:00	Quản trị: mở biểu mẫu sản phẩm, xem danh sách sản phẩm, mở kho	Trạng thái đang bán và đã ẩn phân biệt được; phiếu nhập và tồn thấp cùng màn
12:00	Quản trị: năm báo cáo, đổi khoảng ngày	Doanh thu khớp với số đơn đã thanh toán; biểu đồ có bảng số liệu
14:00	Quản trị: khoá một khách, rồi thử đăng nhập bằng tài khoản đó	Đăng nhập bị từ chối với thông điệp tài khoản bị khoá

Ba mẹo khi trình bày, rút ra từ chính các lần chạy thử: bật chế độ không làm phiền để thông báo hệ thống không che màn hình; giữ máy ảo ở pin 100% và tắt âm thanh; và luôn có video dự phòng đã quay sẵn phòng trường hợp mạng nội bộ gặp sự cố.

B.2. Môi trường phát triển

Bảng B.2. Môi trường của lần chạy cuối

Thành phần	Giá trị
Hệ điều hành	Windows 10 (10.0.26300), x64
Java	Temurin JDK 17
Android SDK	cmdline-tools 22.0, platform-tools 37.0.1, build-tools 35.0.0, platform android-36
Máy ảo	`stylo_api35`, Android 15, 1080×1920 @ 480dpi (360×640dp)
Node.js	24.22
MySQL	8.4 trong container Docker `stylo-mysql`, schema `stylo` và `stylo_test`
Cổng API	8787, nối vào máy ảo bằng `adb reverse tcp:8787 tcp:8787`
Cổng adb	5359, vì dải 4942–5041 bị Windows giữ nên cổng mặc định 5037 không dùng được

B.3. Cách ghi video

Video được ghi ngay trên máy ảo, không quay màn hình máy tính, nên hình ảnh đúng tỉ lệ thiết bị và không lộ cửa sổ của công cụ phát triển.

1. Dựng lại ứng dụng bản debug và cài lên máy ảo. 2. Khởi động server, kiểm tra `/health`, đặt lại cầu `adb reverse`. 3. Bắt đầu `adb shell screenrecord` cho từng đoạn tối đa ba phút, vì công cụ này giới hạn độ dài mỗi tệp; các đoạn được nối lại bằng ffmpeg. 4. Chạy kịch bản ở mục B.1 với nhịp nghỉ khoảng một giây giữa các thao tác để người xem theo kịp. 5. Cắt phần thanh trạng thái ở đầu khung hình để không lộ thông báo hệ thống và đồng hồ, sau đó xuất MP4 một luồng hình.

Bản quay nộp kèm dài 6 phút 51 giây, khung 1080×1792, mở đầu và kết thúc bằng thẻ tiêu đề. Video gồm hai lượt: lượt khách đi từ trang chủ, tìm kiếm không dấu, bộ lọc, chi tiết sản phẩm, đăng nhập, sổ địa chỉ, giỏ hàng, đặt COD tới danh sách đơn; lượt quản trị đi từ đăng nhập, tổng quan, xác nhận đơn đang chờ, danh sách sản phẩm, kho, năm báo cáo tới danh sách người dùng. Đơn trong video là Áo thun basic S/Đen với số lượng năm, tổng 995.000 ₫ và phí giao 0 ₫ vì tạm tính vượt ngưỡng miễn phí. Phần chặn trần tồn được trình bày khi demo trực tiếp vì quay hết một biến thể sẽ làm cạn tồn của dữ liệu mẫu.

Phụ lục C. Các bước chạy lại hệ thống

Phần này đủ để một người khác dựng lại hệ thống từ một máy trống có sẵn Docker, Node.js và Android SDK.

C.1. Cơ sở dữ liệu

```
bash docker run -d --name stylo-mysql -e MYSQL_ROOT_PASSWORD=root \ -e MYSQL_DATABASE=stylo -e MYSQL_USER=stylo -e MYSQL_PASSWORD=stylo \ -p 3306:3306 mysql:8.4 docker exec stylo-mysql mysql -ustylo -pstylo -e "CREATE DATABASE stylo_test"
```

Hai schema được dùng với hai mục đích khác nhau: `stylo` cho dữ liệu phát triển và demo, `stylo_test` cho kiểm thử tích hợp. Kiểm thử tích hợp dừng ngay nếu tên schema không phải `stylo_test`, nên không có cách nào vô tình xoá dữ liệu phát triển.

C.2. API

```
bash cd server npm install cp .env.example .env # điền DATABASE_URL, JWT secrets, PAYMENT_PROVIDER npm run db:reset # migrate + seed npm run dev # API trên 127.0.0.1:8787 curl http://127.0.0.1:8787/health
```

`npm run db:reset` xoá và dựng lại schema rồi chạy seed. Seed có tính tất định: chạy hai lần cho ra cùng tập dữ liệu, cùng thứ tự và cùng SKU. Nếu đổi danh mục seed, phải chạy lại `db:reset` thay vì chỉ `db:seed`, vì thứ tự sinh SKU phụ thuộc thứ tự sinh sản phẩm.

C.3. Ứng dụng Android

```
bash cd android gradlew.bat :app:assembleDebug adb -P 5359 install -r app/build/outputs/apk/debug/app-debug.apk adb -P 5359 reverse tcp:8787 tcp:8787
```

Bản debug trỏ tới `http://127.0.0.1:8787`, tức là địa chỉ nhìn từ trong máy ảo; cầu `adb reverse` biến nó thành kết nối tới máy phát triển. Nếu dùng cổng adb khác, đặt `ANDROID_ADB_SERVER_PORT` cho cả adb, máy ảo và Gradle.

C.4. Tài khoản demo

Bảng C.1. Tài khoản có sẵn trong dữ liệu seed

Vai trò	Email	Mật khẩu
Quản trị	admin@stylo.local	Admin@12345
Khách hàng	customer01@stylo.local … customer12@stylo.local	Customer@12345

Một trong mười hai khách có trạng thái DISABLED để minh hoạ việc khoá tài khoản. Mật khẩu trong dữ liệu seed là mật khẩu demo, không dùng cho bất kỳ môi trường thật nào.

C.5. Chạy kiểm thử

```
bash cd server && npm run test:unit && npm run test:integration
cd android && gradlew.bat :app:testDebugUnitTest android\e2e.cmd
# toàn bộ hành trình Kaspresso android\e2e.cmd -Class
OrdersJourneyTest   # một hành trình
```

Bộ kiểm thử giao diện cần máy ảo đang chạy, API đã lên và cầu adb reverse đã đặt. Script tự kiểm ba điều kiện đó và dừng daemon điều khiển thiết bị trước khi gọi Gradle, vì Kaspresso và daemon đó không thể cùng giữ kết nối UiAutomation.

C.6. Dựng lại hồ sơ

```
bash python docs/report/build_report.py              # sinh
BaoCao_Stylo.docx từ docs/report/content python
docs/report/make_pdf.py          # đo layout hai lượt, ghi số trang
mục lục, xuất PDF python docs/report/tools/erd_from_schema.py --
columns     # schema.prisma -> ERD (mmd, svg, png, A3) python
docs/report/build_slides.py         # sinh Slide_BaoVe.pptx python
docs/report/export_slides.py        # xuất Slide_BaoVe.pdf python
docs/report/tools/make_zip.py       # đóng gói mã nguồn python
docs/report/verify_submission.py # đọc lại các tệp nộp và kiểm
checklist
```

Tệp dump lấy từ container: `docker exec stylo-mysql mysqldump --databases stylo --routines=false --triggers=false > docs/submission/stylo-db.sql`. Bốn script đầu cần Python cùng Word và PowerPoint; script ERD cần Mermaid CLI và một trình duyệt để dựng hình; `verify_submission.py` đọc chính các tệp trong `docs/submission`, nên nó bắt được lỗi thiếu hình, thiếu bảng hoặc sai số trang sau khi dựng lại.